



SAPIENZA
UNIVERSITÀ DI ROMA

Environment-Aware UAV–UGV Networking and Charging Coordination for Disaster Response

Faculty of Information Engineering, Computer Science and Statistics
Corso di Laurea in Applied Computer Science and Artificial Intelligence

Candidate

Liyu Jin

ID number 2050779

Thesis Advisor

Prof.ssa Novella Bartolini

Academic Year 2025/2026

Thesis defended on Not defended yet
in front of a Board of Examiners composed by:

Prof.ssa Novella Bartolini (chairman)

Prof. ...

Prof. ...

Prof. ...

Prof. ...

Prof. ...

Prof. ...

**Environment-Aware UAV–UGV Networking and Charging Coordination for
Disaster Response**

Bachelor's thesis. Sapienza – University of Rome

© 2026 Liyu Jin. All rights reserved

This thesis has been typeset by L^AT_EX and the Sapthesis class.

Author's email: jin.2050779@studenti.uniroma1.it

Acknowledgements

I express my deepest gratitude to my supervisor, **Prof.ssa Novella Bartolini**, for her continuous guidance, patience, and invaluable insights throughout the development of this thesis.

家之所系，心之所安

I am deeply thankful to my parents for their steadfast support throughout my academic journey. I appreciate that they stood by every decision I made, providing not only financial help but also unwavering confidence in me. My mother has always assured me that she would stand behind any path I choose, trusting wholeheartedly in my ability to make sound choices. I am just as grateful for my father's quiet devotion to our family; though he seldom shows his feelings in words, his love is evident in the many small acts that have guided and strengthened me. I am also profoundly grateful to my beloved older sister, whose presence has been a continual source of comfort and resilience. In my hardest times, she has always been there with patience, empathy, and sincere encouragement. She consoled me when I was overwhelmed and spent long hours talking with me, offering insight and companionship precisely when I needed them most.

同舟共济，相伴相惜

I am deeply thankful to all the friends and classmates I've met throughout my university years. To those who spent long hours with me in study rooms, who shared the ups and downs of anxiety and joy, and who stood by my side through endless conversations, short breaks, and much needed moments of relief, thank you for giving me the strength to continue. Your presence turned exhausting days into treasured memories, and your support reminded me that I never had to face this journey on my own.

得之我命，失之我幸

Finally, I want to thank myself the steady and genuine version of me who refused to surrender in moments of uncertainty, who stayed true to her values when it would have been simpler to compromise, and who chose to move ahead with courage whenever obstacles appeared. I am grateful for the strength that carried me through the hardest seasons of my life, and I embrace the idea that whatever comes or goes is part of my path. With this belief, I trust that the road ahead will be just as generous and full of possibility as the one that has led me here.

Words & Signatures

"To the journey we walked together."

Abstract

Unmanned aerial vehicle (UAV) swarms are increasingly considered for rapid situational awareness in disaster response, yet their effectiveness is constrained by unstable aerial networking and limited onboard energy. This thesis presents an environment-aware UAV-UGV cooperative simulator implemented in ROS 2 to study the coupled dynamics of (i) Flying Ad Hoc Network (FANET) robustness and (ii) charging coordination through a ground charging unit (UGV). The simulator models heterogeneous UAV roles (cluster heads and members), multi-hop routing under intermittent connectivity, and energy depletion under environment-driven effects such as weather-dependent communication conditions. A family of charging-queue scheduling policies is designed and integrated, including non-preemptive and preemptive variants (e.g., FCFS, role-priority, EDF, and dynamic score-based policies). Charging requests and decisions are exchanged through the same networked communication substrate used for telemetry, enabling evaluation under degraded connectivity rather than assuming ideal control channels. A structured logging and metrics extraction pipeline is implemented to support reproducible evaluation, capturing per-request charging outcomes and timing, queue and dock utilization time series, UAV battery and survival trajectories, routing events, and recovery behavior after cluster-head failures.

Comparative experiments quantify trade-offs among mission continuity (UAV survival and coverage persistence), network performance (delivery ratio and routing stability), and charging efficiency (waiting time and role-dependent bias). The results provide guidance on how charging scheduling and recovery mechanisms should be co-designed with network conditions to improve resilience in long-horizon disaster monitoring missions.

Contents

1	Introduction	1
2	Background & Literature Review	3
2.1	UAV Communication Networks and FANET Fundamentals	3
2.2	Communication Architectures for FANETs	4
2.3	Routing Protocols in FANETs	4
2.4	Connectivity-Constrained Deployment and Coverage	5
2.5	Energy Constraints and Network Lifetime in FANETs	5
2.6	UAV-UGV Cooperative Systems	6
2.7	Discussion and Research Gap	6
2.8	Robot Operating System 2	7
3	System Architecture and Design	9
3.1	Chapter Overview	9
3.2	System Model and Assumptions	9
3.2.1	Scope of Simulation and Abstractions	9
3.2.2	Time Model and Synchronisation	10
3.2.3	Environment and Weather Model	11
3.2.4	Physical Platform Assumptions and Parameter Grounding	11
3.2.5	Time Compression and Drain-Rate Scaling	13
3.3	Mission Scenario and Deployment Model	14
3.3.1	Disaster Area and Task Model	14
3.3.2	Initial Deployment Strategy	14
3.3.3	Mobility and Patrol Logic	15
3.3.4	Role Assignment and Role Dynamics	15
3.4	FANET Communication and Routing Model	16
3.4.1	Message Model and Shared Communication Substrate	16
3.4.2	Link and Impairment Model	16
3.4.3	Routing Model	16
3.4.4	Network Monitoring Hooks	17
3.5	UAV Agent Design	17
3.5.1	UAV State Machine	17
3.5.2	Energy Dynamics in UAV Behaviour	18
3.5.3	Charging Request Generation	19
3.6	UGV Charger and Scheduling Policies	19
3.6.1	UGV Charger Model	19
3.6.2	Scheduling Policy Definitions	19
3.6.3	Decision Outputs and Protocol Semantics	20
3.7	Status, Heartbeat, Failure Detection, and Recovery	21
3.7.1	Status Reporting Model	21

3.7.2	Heartbeat and Liveness Detection	21
3.7.3	Recovery Workflows	21
3.8	ROS 2 Implementation View	22
3.8.1	Package and Node Organisation	22
3.8.2	Node–Topic/Service/Action Interface Table	22
3.8.3	Execution Model and Configuration	22
3.9	Experiment Configuration and Data Logging Design	23
3.9.1	Run Configuration and Reproducibility	23
3.9.2	Logged Data Products	23
3.9.3	Metric Extraction Pipeline Preview	23
4	Experiment & Results	24
4.1	Experimental Design and Evaluation Methodology	24
4.1.1	Experimental Objective and Research-Question Mapping	24
4.1.2	Protocols Under Comparison	24
4.1.3	Replication Strategy, Dataset Scope, and Artifact Availability	25
4.1.4	Evaluation Metrics	25
4.1.5	Dataset Reliability Note	26
4.2	Aggregate Protocol-Level Results	26
4.2.1	Overall Network Performance	26
4.2.2	Charging Scheduler Performance	27
4.2.3	Fleet Survivability	28
4.2.4	Aggregate Trade-Off View	29
4.3	Temporal Dynamics of Charging and Network Quality	30
4.3.1	PDR Evolution Over Time	30
4.3.2	Queue Pressure, Dock Utilisation, and Charging Throughput	30
4.3.3	Weather Context and Time-Varying Degradation	31
4.4	Cross-Layer Analysis: From Scheduling to Network Quality	31
4.4.1	Statistical Association Between Charging Outcomes and PDR	35
4.4.2	Mechanism Interpretation	35
4.4.3	Why Request Reachability Matters More Than Decision Speed	36
4.4.4	The CH Priority Paradox: Why Role-Based Protocols Underperform	38
4.5	Delay Interpretation and Measurement Bias	39
4.6	Controlled Validation Under Fixed Weather	41
4.6.1	Rationale and Experimental Setup	42
4.6.2	Results and Comparison with Test Round 2	42
4.6.3	Interpretation	42
5	Conclusions & Future Work	43
	Appendix Guide	44
A	Message and Protocol Specifications	45
A.1	TrafficMessage schema	45
A.2	Charging decision payload specification	45
A.3	ROS 2 package and interface tables	46

B Algorithms and Mathematical Details	48
B.1 Deployment algorithms	48
B.1.1 Cluster-head placement from task points (k-means)	48
B.1.2 Connectivity refinement	48
B.1.3 UGV placement via geometric median (Weiszfeld algorithm)	48
B.2 Weather impairment and energy drain models	49
B.2.1 Packet-drop model in the fault injector	49
B.2.2 Weather-dependent energy drain model	49
C Parameter Tables and Calibration Details	50
C.1 Time compression and parameter mapping	50
C.1.1 Time-compression factor	50
C.1.2 Core simulation parameters	50
C.1.3 Implied effective endurance	50
D Logging Schema and Metric Extraction Details	52
D.1 Logging schema	52
D.2 Metric extraction details	53
E Data analysis details	54
E.1 Metric Definitions	54
E.1.1 Network metrics	54
E.1.2 Charging metrics	54
E.1.3 Survivability metrics	55
E.2 Aggregation and Statistical Summaries	55
E.2.1 Mean and standard deviation	55
E.2.2 Coefficient of variation	55
E.2.3 Correlation summaries	55
E.3 Windowing and Time-Series Processing	56
E.3.1 Valid-sample filtering	56
E.3.2 Merged time-series plots	56
E.3.3 Weather joins	56
E.3.4 Event binning	56
E.4 PDR Dip Threshold for Event-Aligned Plots	56
E.5 Delay Robustness and Bias-Corrected Delay	56
E.6 Dataset Validation and Preprocessing Notes	57
E.6.1 Dataset Completeness and Structural Consistency	57
E.6.2 Column-Level Caveats and Sentinel Handling	57
E.6.3 Workload Comparability Across Protocols	57
E.6.4 Duplicated Windows and Non-Corrupt Anomalies	57
E.6.5 Role-Stratified Charging KPI Computation	58
E.6.6 Cross-CH Request Synchronization Analysis	58
Bibliography	59

Chapter 1

Introduction

Floods are among the most disruptive natural hazards for modern societies: they can span wide geographic areas, persist for long periods, and severely damage critical infrastructure. Unlike localized emergencies, floods may simultaneously compromise roads, power distribution, and terrestrial communication facilities, complicating both rescue operations and real-time monitoring. A recent and emblematic case is the **2023 Emilia-Romagna flood in Italy**, which caused extensive damage across urban and rural areas and was accompanied by prolonged disruptions to ground communication networks [1]. In these conditions, maintaining situational awareness and ensuring reliable information flow becomes a prerequisite for effective emergency coordination [2].

Unmanned Aerial Vehicles (UAVs) have therefore become an important asset in disaster response, offering rapid deployment, flexible sensing, and independence from damaged ground infrastructure. In flood scenarios, UAVs are widely used for aerial surveillance, damage assessment, victim localization, and environmental monitoring. However, their operational value depends not only on sensing quality but also on the ability to deliver time-sensitive data reliably. Flood response is inherently delay-critical: late, missing, or inconsistent information can translate into poor decisions and slower interventions [3].

To scale monitoring to large affected areas, recent research increasingly shifts from single-UAV missions to *multi-UAV networks*, commonly referred to as Flying Ad Hoc Networks (FANETs) [4]. In this paradigm, UAVs form a coordinated wireless system that forwards telemetry over multi-hop links toward a sink or a ground control station. Yet FANET reliability remains challenging due to rapid topology changes, limited communication ranges, and stringent latency requirements [5]. These constraints are amplified in disaster contexts where mobility patterns are dictated by coverage needs and where network partitions may occur frequently. These networking challenges are compounded by energy constraints that limit mission duration — a gap that UAV-UGV cooperation is specifically designed to close. Long-duration operations further expose the limitations of UAV-only solutions: restricted flight time forces repeated mission interruptions and can induce temporary loss of monitoring continuity and network fragmentation [3, 6]. A promising mitigation is *UAV-UGV cooperation*, where Unmanned Ground Vehicles (UGVs) act as mobile charging stations and stable ground support nodes. In flood-affected environments, UGVs can exploit partially accessible terrain to provide energy replenishment and extend aerial mission duration, improving overall operational resilience.

Motivated by these observations, this thesis studies a *UAV-UGV cooperative monitoring system for flood-inspired disaster scenarios* with emphasis on the joint effects of FANET robustness and charging coordination. The proposed system

adopts a role-based multi-UAV architecture supported by a mobile UGV charger and is implemented in a ROS 2-based simulation framework. This framework enables comparative evaluation of communication resilience, monitoring continuity, and energy-aware scheduling under realistic conditions inspired by the 2023 Emilia-Romagna event.

Research objectives. This thesis evaluates how networking constraints and energy replenishment mechanisms jointly shape long-horizon disaster monitoring performance. The focus is on the interaction between FANET robustness and UGV-based charging coordination under intermittent connectivity and environment-driven effects.

Research questions. The work is guided by the following questions:

- **RQ1:** How do intermittent connectivity and cluster-head disruptions impact end-to-end telemetry delivery and routing stability in a flood-oriented FANET?
- **RQ2:** How do charging-queue scheduling strategies influence mission continuity (e.g., monitoring persistence), and what role-dependent bias emerges between cluster heads and members?
- **RQ3:** What trade-offs arise when charging control traffic must traverse the same networked communication substrate as telemetry, rather than relying on an idealized out-of-band control channel?

Contributions. The main contributions of this thesis are:

- A ROS 2-based UAV-UGV cooperative simulation framework for flood-inspired disaster monitoring that models multi-hop FANET communication, heterogeneous UAV roles, and environment-driven effects.
- Integration and comparison of multiple charging-queue scheduling policies, including non-preemptive and preemptive variants, with charging requests/decisions exchanged through the same networked communication layer used for telemetry.
- A structured logging and metrics pipeline enabling reproducible analysis of network performance, charging efficiency, and recovery behavior under failures.

The remainder of this thesis is organized as follows:

Chapter 2 reviews the relevant literature on UAV communication networks and disaster response.

Chapter 3 presents the proposed system architecture.

Chapter 4 describes the simulation setup and experimental evaluation.

Chapter 5 concludes the thesis and outlines future research directions.

Chapter 2

Background & Literature Review

2.1 UAV Communication Networks and FANET Fundamentals

Unmanned Aerial Vehicles (UAVs) have increasingly been adopted as communication platforms in scenarios where terrestrial infrastructure is unavailable, damaged, or unreliable, such as natural disasters and emergency response operations. When multiple UAVs cooperate to exchange data and forward information toward a ground station or sink node, they form a *Flying Ad Hoc Network* (FANET), a class of wireless networks characterized by high node mobility, rapidly changing topology, and predominantly line-of-sight communication links [4, 5].

As highlighted in recent surveys, these characteristics lead to short link lifetimes, intermittent connectivity, and increased routing overhead, making conventional ad hoc networking protocols insufficient for UAV-based systems without adaptation [7, 8].

Communication in FANETs is commonly organized in a multi-hop fashion, where UAVs act both as sensing platforms and as relay nodes to extend network coverage beyond the communication range of a single device. This capability is particularly relevant in disaster scenarios, where direct communication between all UAVs and a ground control station cannot be assumed. Multi-hop communication enables scalable monitoring over large areas but introduces additional challenges related to latency, routing stability, and network partitioning [2].

Several architectural paradigms for FANETs have been proposed in the literature, including centralized, decentralized, and hierarchical approaches. Centralized architectures rely on a single control entity, which simplifies coordination but introduces a single point of failure and limits scalability. Decentralized approaches improve robustness but often suffer from increased coordination overhead and reduced global awareness. Hierarchical architectures, where a subset of UAVs assumes coordination or relay roles, have been proposed as a compromise to improve scalability and manageability while preserving network connectivity [4, 7].

Despite extensive research on FANET communication architectures, maintaining reliable connectivity remains a fundamental challenge, especially in safety- and delay-critical applications. Dynamic topology changes, limited transmission ranges, and strict latency requirements impose strong constraints on deployment and routing strategies.

2.2 Communication Architectures for FANETs

FANET communication architectures are broadly categorised as centralised, distributed, and hierarchical. Centralised designs enable global optimisation but introduce a single point of failure and scale poorly under dynamic topologies [5]. Distributed designs improve resilience at the cost of coordination overhead [7]. Hierarchical designs assign relay and coordination roles to a subset of UAVs, improving scalability and multi-hop forwarding efficiency — but cluster-head failure can severely degrade network performance, making robust role management essential [4, 3].

2.3 Routing Protocols in FANETs

Routing is a fundamental component of Flying Ad Hoc Networks (FANETs), as it determines how sensing data and control messages are forwarded across highly dynamic aerial networks. In disaster monitoring scenarios, routing protocols must operate under strict latency requirements, intermittent connectivity, and frequent topology changes. Unlike traditional ad hoc networks, FANETs are characterized by high node mobility, three-dimensional movement, and short link lifetimes, which significantly affect routing performance [4, 5]. Existing routing solutions for FANETs can be broadly classified into topology-based, geographic, hierarchical, and connectivity-aware approaches.

Topology-Based Routing Protocols Topology-based routing protocols rely on explicit route discovery and maintenance mechanisms to establish end-to-end communication paths. Representative examples include adaptations of Mobile Ad Hoc Network (MANET) protocols such as the Ad hoc On-Demand Distance Vector (AODV) and the Optimized Link State Routing (OLSR) protocol, which have been modified to account for UAV mobility [7].

Geographic and Position-Based Routing Geographic routing protocols exploit location information, typically obtained via Global Navigation Satellite Systems (GNSS), to make forwarding decisions based on node positions rather than explicit routes. Protocols such as Greedy Perimeter Stateless Routing (GPSR) and its UAV-oriented variants forward packets toward the neighbor geographically closest to the destination [5, 9].

Hierarchical and Cluster-Based Routing Hierarchical routing protocols organize UAVs into clusters and assign specific roles, such as cluster heads or relay nodes, to manage routing and coordination within the network. Data generated by cluster members is typically forwarded to cluster heads, which then relay information toward the sink through inter-cluster communication [4, 3].

Delay- and Connectivity-Aware Routing In disaster response applications, routing protocols must often satisfy delay constraints while preserving network connectivity. Delay- and connectivity-aware routing approaches explicitly consider latency requirements and link availability during routing decisions. These protocols may adapt UAV positions, transmission power, or forwarding strategies to ensure timely data delivery [3, 6].

In addition, many existing solutions focus on specific aspects of routing or deployment and are evaluated under simplified conditions, reducing their applicability to realistic disaster monitoring missions where intermittent links, three-dimensional mobility, and energy constraints act simultaneously.

Overall, topology-based and geographic protocols offer simplicity but struggle with high mobility and frequent route breakage. Hierarchical routing improves reliability and scalability but depends on cluster-head stability — a vulnerability directly relevant to this thesis. Delay- and connectivity-aware approaches address latency requirements but assume stronger network control. These limitations motivate routing strategies co-designed with deployment planning and energy management, as explored in this work.

2.4 Connectivity-Constrained Deployment and Coverage

Early multi-UAV deployment strategies primarily focused on coverage maximization, often adopting grid-based, formation-based, or target-oriented placement methods. While these approaches can efficiently distribute sensing resources over a given area, they typically neglect connectivity constraints, implicitly assuming that communication links will remain available [4]. In practice, especially in infrastructureless and dynamic environments, coverage-oriented deployments frequently lead to network partitioning, increased latency, and packet loss, undermining the effectiveness of monitoring operations.

To address these limitations, connectivity-constrained deployment approaches explicitly incorporate communication requirements into UAV positioning decisions. These methods aim to guarantee that the resulting spatial configuration preserves network connectivity, often by enforcing distance constraints between neighboring UAVs or by strategically positioning relay nodes [3]. In delay-critical applications, deployment strategies may further account for end-to-end latency bounds, ensuring that information can be delivered within acceptable time limits. Connectivity-aware deployment strategies have been shown to improve network robustness and predictability, especially in safety-critical scenarios. However, they often rely on simplified mobility assumptions or static network configurations, limiting their applicability to long-duration missions with dynamic conditions. Furthermore, enforcing connectivity constraints may increase UAV mobility and hovering time, leading to higher energy consumption and reduced network lifetime. These observations suggest that connectivity-constrained deployment must be integrated with energy-aware mechanisms to support sustained disaster monitoring operations. These observations directly motivate the integration of external energy replenishment into the deployment model, addressed in the following sections.

2.5 Energy Constraints and Network Lifetime in FANETs

UAV energy consumption arises from multiple sources, including propulsion, hovering, sensing, and wireless communication. Among these, mobility-related energy costs typically dominate, especially in multi-UAV deployments where frequent repositioning is required to maintain coverage or connectivity. Communication overhead, such as control message exchange and multi-hop forwarding, further contributes to battery depletion, particularly in dense or highly dynamic FANETs [4]. These factors create a strong interdependence between network operation and

energy expenditure.

A further challenge arises from the uneven energy consumption across the network. UAVs acting as relays, cluster heads, or communication gateways tend to deplete their batteries faster than peripheral sensing nodes. The failure of these critical nodes can lead to network fragmentation, increased latency, or complete loss of connectivity, even when other UAVs still have sufficient energy. This imbalance highlights the limitations of UAV-only FANETs in maintaining stable communication structures over long mission durations [3].

2.6 UAV-UGV Cooperative Systems

The endurance limitations of UAV-only Flying Ad Hoc Networks (FANETs) motivate cooperative architectures that combine Unmanned Aerial Vehicles (UAVs) with Unmanned Ground Vehicles (UGVs). In disaster monitoring scenarios, UGVs can complement aerial platforms by providing persistent ground mobility, higher payload capacity, and more stable long-term operation, enabling cooperation for sensing support, communication assistance, and energy replenishment [10, 2].

UGVs as mobile recharging stations. A widely adopted role for UGVs is to operate as mobile charging stations that enable in-field battery replenishment. This reduces the frequency of forced mission interruptions due to battery depletion and can extend monitoring duration in long-horizon disaster scenarios where fixed charging infrastructure is unavailable [10, 11, 12]. When multiple UAVs share limited charging capacity, the effectiveness of UAV-UGV cooperation depends not only on the feasibility of rendezvous but also on how charging requests are prioritized and served. Queueing and scheduling strategies influence waiting time, throughput of the charging dock across heterogeneous UAV roles. In flood monitoring FANETs, this aspect is especially important because charging interruptions can trigger temporary network fragmentation and degrade telemetry delivery [3, 6].

UGVs as communication support nodes. Beyond energy support, UGVs may also serve as stable ground nodes for data aggregation or as communication relays, potentially improving end-to-end connectivity when aerial topology becomes sparse or fragmented [10, 5]. The benefit of this approach depends on the integration of ground support nodes into the overall communication architecture and on the dynamics of the multi-hop aerial network [4].

2.7 Discussion and Research Gap

Existing UAV-UGV cooperative systems provide a promising direction for mitigating endurance limitations, but leave a significant integration gap. Most prior work focuses on vehicle-level rendezvous feasibility or task-specific planning [10, 11, 12], and does not examine how charging coordination interacts with FANET robustness when charging control traffic must traverse the same multi-hop aerial network as telemetry. This gap has three concrete dimensions. First, it is unclear how intermittent connectivity and cluster-head disruptions affect end-to-end telemetry delivery and routing stability in a flood-oriented FANET (RQ1). Second, the operational impact of different charging scheduling strategies on mission continuity — and the role-dependent bias they introduce between cluster heads and members — has not been characterised (RQ2). Third, the trade-offs that arise when charging control shares

the communication substrate with telemetry, rather than relying on an idealised out-of-band channel, remain unexplored (RQ3). This thesis addresses all three dimensions through a ROS 2-based simulation framework described in the following chapter.

2.8 Robot Operating System 2

Robot Operating System 2 (ROS 2) is an open-source software framework (not a conventional operating system) that provides libraries, tools, and conventions for building modular robot applications and distributed robotic systems [13, 14, 15]. In our simulator, the software is organized as a ROS 2 workspace composed of multiple packages and nodes implementing UAV/UGV behaviors and supporting services.

ROS graph: nodes and communication interfaces. A ROS2 system can be viewed as a graph of nodes, computational participants that communicate with other nodes in the same process, a different process, or on a different machine [15, 16]. The nodes interact through standardized interfaces, primarily topics, services, actions, and parameters [15, 17, 18, 19, 20].

Topics (publish/subscribe). Topics implement a publish/subscribe communication pattern where publishers and subscribers exchange strongly-typed messages by matching on a shared topic name [17]. This decouples data producers and consumers and naturally supports many-to-many communication for continuous data streams (e.g., robot state, telemetry, sensor data) [17].

Services and actions. Services provide request-reply (remote procedure call) interactions where a client requests a computation and receives a single response [18]. Actions are intended for long-running tasks: they provide goal management, feedback during execution, and support cancellation/preemption semantics [19].

Parameters. ROS 2 parameters are node-associated configuration values used to tune node behavior at startup and, optionally, at runtime without changing code. In the simulator, parameters are used to configure UAV roles, energy thresholds, charging policy selection, and communication range at launch time, allowing controlled experimental variation without modifying node source code.

Execution model: callbacks and executors. ROS 2 is event-driven: subscriptions, timers, and service/action events trigger callbacks. Execution management is handled by executors, which use one or more OS threads to schedule and invoke callbacks for subscriptions, timers, service servers, action servers, and other entities [22].

Communication middleware: DDS/RTPS and the RMW abstraction. ROS 2 supports multiple middleware implementations (initially DDS/RTPS-based), which provide discovery, serialization, and transport for distributed communication [23, 24]. The `rmw` (ROS Middleware) API forms the interface between the ROS 2 software stack and the underlying middleware implementation [25]. Concrete ROS 2 middleware implementations (e.g., DDS- or non-DDS-based) are provided via `rmw_` packages that integrate external protocols with the ROS 2 middleware API [26].

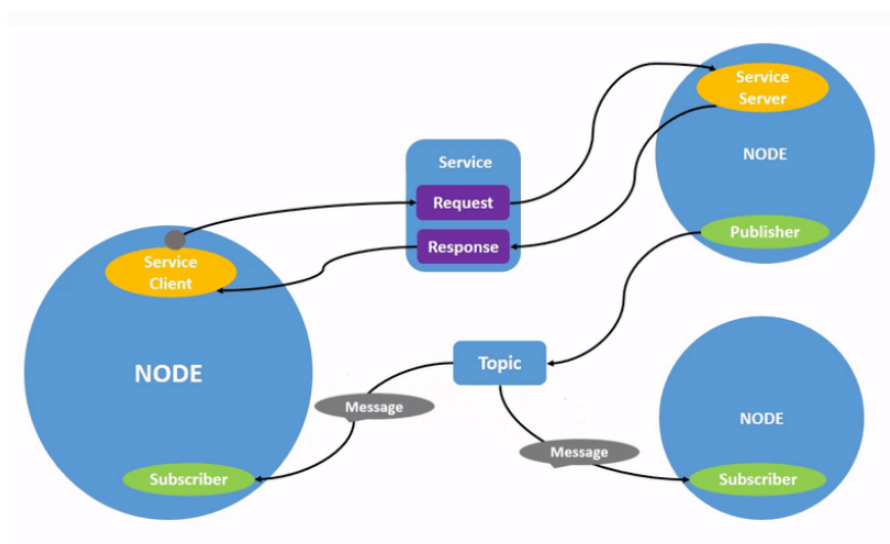


Figure 2.1. ROS 2 node communication via topics and services [21].

Chapter 3

System Architecture and Design

3.1 Chapter Overview

This chapter presents the complete system model, design decisions, and implementation structure of the ROS 2-based UAV–UGV cooperative simulator developed for this thesis.

The simulator codebase, accompanying technical reports, and all raw data collected throughout the experiments are publicly available in the project repository at https://github.com/kk-Syuer/UAV-UGV_monitoring.

3.2 System Model and Assumptions

3.2.1 Scope of Simulation and Abstractions

The simulator models the following phenomena explicitly:

- **Flying Ad Hoc Network (FANET) overlay.** All inter-agent packets — telemetry, coordination commands, charging requests and decisions, and recovery control — are encapsulated as `uav_msgs/msg/TrafficMessage` and routed hop-by-hop through an application-layer two-stage bus (`/fanet/network_bus_raw` → `/fanet/network_bus`). The fault-injector node sits between the two stages and applies weather-dependent packet drops.
- **UAV roles and cluster structure.** Each UAV is either a Cluster Head (CH, `role=1`) or a Member (`role=0`). Only CHs participate in backbone forwarding; Members generate telemetry and request charging but do not forward packets.
- **Charging queue and dock model.** The UGV maintains a waiting queue and a set of concurrently charging UAVs bounded by a configurable dock capacity (`ugv.max_parallel_spots`). Four non-preemptive and three preemptive scheduling policies are implemented and selected at run time via YAML.
- **Weather-driven impairment and energy drain.** A weather server publishes a global regime (`/environment/weather`). The fault injector translates wind speed, rain intensity, and temperature deviation into a packet-drop probability; the UAV nodes read the same topic to adjust their energy consumption rate.

- **Failure detection and recovery.** A dedicated `recovery_manager_node` acts as a watchdog that detects CH timeouts or battery-dead failures and injects recovery control messages (`RECOVERY_START`, `CLUSTER_REASSIGN`, `TASK_ASSIGN`, `NEW_DEPLOYMENT`, `MEMBER_FALLBACK`, `RECOVERY_DONE`) through the FANET overlay.
- **Structured logging.** A dedicated `network_monitor_node` subscribes to all relevant topics as an omniscient observer and writes per-run CSV and JSON artefacts under `<output_dir>/<run_id>/`.

The following aspects are deliberately abstracted away to keep the simulator tractable:

- **Aerodynamics and flight dynamics.** No aerodynamic drag, wind-induced drift, or rotor dynamics are modelled.
- **Photogrammetry and sensor payloads.** The simulator represents sensing coverage only through task-point visitation; raw imagery, point-cloud generation, and image processing pipelines are outside scope.
- **Physical radio propagation.** Link quality is abstracted to a binary reachability check based on Euclidean distance against a configurable communication radius, supplemented by a probabilistic drop layer driven by weather. No multipath, path loss exponents, or antenna patterns are modelled.
- **UGV navigation and path planning.** The UGV charger node moves toward a target pose using the same kinematic step controller as the UAV, without obstacle avoidance or terrain-aware path planning.

3.2.2 Time Model and Synchronisation

All nodes share the system wall clock, which acts as the common time base for both simulation and logging. The following periodic update frequencies are active at runtime:

- **FANET network tick.** UAV nodes publish traffic messages on a 2-second period (`publishTraffic` timer). Heartbeat and status beacons are published every 1 second.
- **Status tick.** `/fanet/status` beacons from UAVs, the UGV, and the sink are published at 1 Hz, governing routing freshness and failure detection.
- **Weather tick.** The weather node publishes `/environment/weather` at a configurable `update_period_sec` (default as configured per run YAML); regime transitions are driven by a separate `macrostate_period_sec` timer.
- **Logging tick.** The network monitor flushes its in-memory records to disk every `csv_write_period_sec` (default 10 s). Time-series rows (status, weather, queue, network) are written every `status_sample_period_sec` (default 1 s).

Reproducibility between runs of the same configuration is achieved by fixing the global RNG seed (`global.rng_seed`), the weather seed (`weather.seed`), and the task-point generation seed; these are passed from the YAML run configuration through the launch file to each node's parameter set.

3.2.3 Environment and Weather Model

The weather subsystem is implemented in `weather_node` (`src/weather_server/src/weather_node.cpp`) and supports two modes:

- **Fixed mode** (`weather.mode: fixed`). The regime is held constant for the entire run. Valid regime strings are `sunny`, `cloudy`, `windy`, `rainy`, and `stormy`.
- **Markov mode** (`weather.mode: markov`). A configurable 5×5 transition matrix governs stochastic regime transitions on a `macrostate_period_sec` timer. Rows are normalised automatically. The seed parameter controls the pseudo-random sequence.

For each regime the node publishes numeric fields: `temperature_c`, `wind_speed`, `wind_direction_deg`, and `rain_intensity` on `/environment/weather`.

Two subsystems consume this topic:

1. **Fault injector.** The fault injector translates wind, rain, and temperature deviation into a probabilistic packet-drop process applied to traffic transiting from `/fanet/network_bus_raw` to `/fanet/network_bus`. Separate scaling factors allow control and data traffic to be impaired at different rates. The full drop model and its parameters are provided in Appendix B.2.
2. **UAV nodes.** UAV agents adjust their energy consumption rate under adverse weather conditions, so wind and rain accelerate battery depletion and increase the frequency of charging requests. The detailed drain-rate model (including wind, rain, and temperature multipliers) is provided in Appendix B.2.

In the first set of experiments presented in this thesis, the simulator is run in Markov mode with the initial state set to *rainy*. The transition probabilities, illustrated in Figure 3.1, are deliberately skewed toward adverse conditions — with *rainy*, *stormy*, and *windy* collectively dominating the stationary distribution — so as to reflect the prolonged degraded-weather conditions typical of flood disaster scenarios. Given that the simulator abstracts radio propagation and flight dynamics, we ground the remaining free parameters—communication range, battery energy, charging time/power, and initial co-deployment assumptions—against publicly documented commercial UAV specifications and recent experimental networking studies.

3.2.4 Physical Platform Assumptions and Parameter Grounding

Platform parameters are grounded against commercial multirotor UAVs that represent the prosumer and enterprise classes typically deployed in search, inspection, and disaster-response missions. Key reference platforms are the DJI Mavic 3 Classic (77 Wh, 46 min advertised flight time [27]), the DJI Air 3 (62.6 Wh), and the DJI Matrice 350 RTK enterprise platform (263.2 Wh TB65 pack). Energy is tracked in the simulator as an absolute quantity in watt-hours; the displayed battery percentage is derived from that value divided by the configured capacity. Drain rates are chosen so that, under nominal weather conditions (no wind, moderate temperature), the simulated endurance matches the published flight-time order of magnitude for each class.

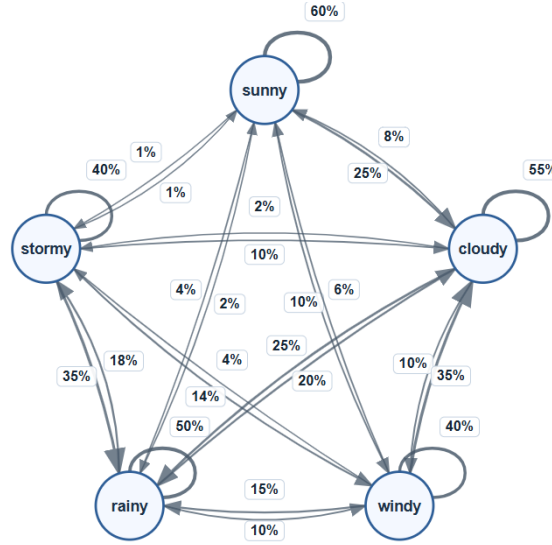


Figure 3.1. Weather Markov transition graph used in the simulator. Transition probabilities are biased toward adverse states (*rainy*, *stormy*, *windy*) to model flood-scenario conditions.

Communication radius. The parameter `network.comm_radius_m` (set to 400 m in all experiments) represents the *effective reliable link radius* for UAV-to-UAV multi-hop ad-hoc forwarding of telemetry packets - including payloads of the order of hundreds of bytes such as sensor readings and imagery metadata - and is not the marketing range of proprietary point-to-point video links. Achievable range in practice depends on radio technology, antenna gains, line-of-sight conditions, and required throughput; heavier telemetry traffic typically reduces reliable range compared to low-rate control signals. For Wi-Fi-class UAV links (IEEE 802.11), research has demonstrated reliable communication up to several hundred meters in open-area conditions [28], and multi-hop relay strategies are explicitly required when swarms must cover larger areas [29]. The 400 m value therefore represents a conservative operating radius for a Wi-Fi/mesh FANET, and is further conservative because the simulator also applies weather-driven packet loss on top of the reachability model (Section 3.2.3). This is distinct from proprietary long-range video links such as DJI O3/O4, whose multi-kilometre advertised ranges apply to single-pair video streaming under ideal conditions and are not representative of multi-hop ad-hoc forwarding of telemetry at the network layer.

Battery capacity, endurance, and role-based drain. The member UAV battery capacity is set to 77 Wh, matching the DJI Mavic 3 Classic Intelligent Flight Battery [27]; the cluster-head capacity is set to 115.2 Wh to represent a larger-frame platform with greater energy budget. The advertised hover endurance of the Mavic 3 Classic is 40 minutes under controlled windless conditions at sea level [27]; real-world mission endurance for prosumer-class multirotors falls in the 30–46 minute range depending on payload, wind, and temperature. Cluster-head UAVs carry the additional burden of multi-hop packet forwarding on the backbone, and therefore operate at a higher drain rate (0.279 Wh/s) than members (0.223 Wh/s), reflecting both relay processing overhead and the assumption of a slightly heavier platform. Both capacity and drain values are exposed as YAML parameters and are consistent across all nodes via the shared `battery` block in the run configuration.

Charging turnaround and dock model. Industrial dock-based charging systems use partial recharge windows rather than full 0–100% cycles to minimise return-to-service time. The DJI Dock 2 specification reports a recharge from 20% to 90% in 32 minutes at 25 °C [30], setting a well-established industry reference for the minimum interval between consecutive sorties. The simulator adopts a similar operational rationale: the UGV charger provides a finite number of parallel docking slots (`ugv.max_parallel_spots` = 3), UAVs enter the queue before capacity is fully depleted, and scheduling policies determine which UAV is served next. Charging is interrupted or reordered under preemptive policies, mirroring scenarios where a critically low UAV must displace a partially charged one.

Mothership co-deployment abstraction. The simulator initialises member UAVs co-located with their assigned cluster-head at deployment time, modelling a *mothership-inspired* logical co-deployment in which members are released from a common launch point. This abstraction captures coordinated launch timing and initial spatial correlation without modelling physical docking hardware, aerodynamic coupling, or payload-induced energy cost on the carrying platform; these effects are explicitly out of scope. The feasibility of physically transporting smaller UAV units is supported by the payload capabilities of industrial-grade platforms: the DJI FlyCart 30 heavy-lift UAV, for example, supports up to 30 kg of payload in dual-battery mode and 40 kg in single-battery mode [31], far exceeding the mass of any prosumer-class multirotor used as a member in this work.



Figure 3.2. DJI FlyCart 30 heavy-lift UAV in cargo mode, capable of carrying up to 30 kg (dual-battery) or 40 kg (single-battery) [31]. This class of platform motivates the mothership co-deployment abstraction: payload capacities far exceed the mass of any prosumer-class member UAV, making coordinated launch from a common carrier physically plausible.

3.2.5 Time Compression and Drain-Rate Scaling

Experiments are executed for a wall-clock simulation horizon of three hours, yet are designed to represent longer-horizon operational dynamics. We therefore apply a time-compression factor, and scale energy drain and charging power consistently so that relative charge-drain dynamics are preserved while the simulated endurance remains in the order of magnitude of commercial multirotor flight times.

All concrete parameter values and the implied endurance mapping under time compression are reported in Appendix C.1. Throughout the results chapter, time axes are shown in simulation time.

3.3 Mission Scenario and Deployment Model

3.3.1 Disaster Area and Task Model

The mission scenario is over a partially accessible terrain, motivating the choice of a 2-D area with configurable bounds ($x_{\min}$, $x_{\max}$, $y_{\min}$, $y_{\max}$). Within this area, a set of *task points* represents sensing or monitoring targets that must be visited by UAVs.

Task points are managed by `coverage_planner_node` and published on `/coverage_planner/task_points` with transient-local QoS, so that any late-joining node (including the visualiser) receives the full set. Each `TaskPoint` carries an identifier, a cluster assignment string, and a 3-D position.

The *sink* (`sink_gateway_node`) acts as the ground station and final telemetry destination. It is assigned a fixed pose during initial deployment. The UGV (`ugv_charger_node`) is also deployed to a fixed initial pose within the area, from which it can move to optimise charging rendezvous.

3.3.2 Initial Deployment Strategy

Initial deployment is computed centrally by `coverage_planner_node`, which places the sink and UGV, assigns UAV roles and clusters, and publishes deployment commands through the FANET overlay. Each UAV receives its target pose, cluster assignment, and initial next-hop hints via the message payload. After all expected deployment acknowledgements are received, the sink issues a `MOTION_START` control message that gates the beginning of mission mobility.

Task-point generation modes. Task points are generated according to

`taskpoint_generation_mode`, parsed in `parseTaskPoints()`. Three modes are supported, with an automatic fallback to random generation if an unrecognised mode is specified:

- **random.** A random number of task points in the range $[5, 15]$ is drawn, and each point (x, y) is sampled uniformly within the planner bounds. The planner RNG is seeded by `rng_seed`; a negative value causes the seed to be drawn from `std::random_device`, yielding a non-reproducible layout.
- **fixed_file.** Task points are loaded from the `taskpoints:` field of a YAML file specified by `fixed_taskpoints_file`. Coordinates are validated as numeric and clamped to the planner bounds; the list is optionally truncated to `fixed_taskpoints_count` entries. This mode is used in all canonical experiments to guarantee identical task layouts across runs with different charging policies.

CH placement algorithm. Task points are partitioned into N_{CH} clusters and each cluster-head is assigned a target pose derived from its cluster geometry. A lightweight clustering approach is sufficient here because the fleet size and the number of task points are small and fixed across experiments. Full algorithmic details (including the update rule and iteration budget) are provided in Appendix B.1.

Sink and UGV placement. The sink gateway is fixed at the origin $(0, 0)$. The UGV is placed near a robust central location with respect to the cluster-head positions, to reduce the worst-case rendezvous distance for charging and to avoid

sensitivity to outlier clusters. The geometric-median formulation and the Weiszfeld-style solver used in the simulator are documented in Appendix B.1. Both placements are published as deployment commands through the same FANET overlay used for UAV deployment, ensuring consistent handling across all node types.

3.3.3 Mobility and Patrol Logic

Member UAVs (role 0) begin patrol only after two conditions are met: the `DEPLOYMENT_CMD` has been acknowledged and the sink has broadcast `MOTION_START`. Until then, members remain co-located with their CH (`syncPoseToCh`), and do not enter task mobility until the CH itself reaches its deployment pose and issues a `TASK_RELEASE` message.

Once released, the member builds a patrol path over its assigned task points using a nearest-neighbour TSP heuristic (`buildTspPath`), falling back to random waypoints within the CH service radius if no task points are assigned. The member then executes the path continuously — moving point-to-point via `stepTowards2D` at `uav_speed_mps_`, generating telemetry at each waypoint, and looping back to the first point on completion. Patrol is temporarily suspended in two cases: when a charge request has been sent but no `CHARGE_DECISION` has been received yet, the member moves toward a CH rendezvous point while waiting; and when buffered telemetry must be uploaded, the member detours briefly toward the CH before resuming its route. Other blocking conditions (battery depletion, active charging, emergency landing) are handled by the charging FSM described in Section 3.5.1.

CH UAVs don't execute the same patrol and telemetry logic as members, they perform backbone forwarding duties, relaying FANET traffic for all members in their cluster using next-hop entries from the routing table.

3.3.4 Role Assignment and Role Dynamics

Roles are assigned statically at launch time through the YAML configuration and are embedded in each UAV's `uav_id` and `role` parameter. In the baseline simulator design, role assignment does not change during normal operation; it may change as a result of recovery actions (see Section 3.7).

CHs are responsible for:

- forwarding multi-hop FANET traffic (backbone routing),
- acting as the first hop for their cluster's Members when those Members send telemetry or charging requests,
- consuming routing table entries to determine next hops toward the sink and the UGV.

Members are responsible for:

- generating periodic telemetry (`flow_type=0` DATA messages) addressed to the sink,
- sending `CHARGE_REQUEST` control messages to the UGV when their battery level falls below the configured threshold,
- obeying `CHARGE_DECISION` messages and transitioning through the charging state machine.

The role assignment directly affects charging priority in role-aware scheduling policies (Section 3.6.2): CHs are assigned a higher priority weight in `role_priority` and `dynamic` policies because the loss of a CH disrupts backbone connectivity for all of its cluster’s Members.

3.4 FANET Communication and Routing Model

3.4.1 Message Model and Shared Communication Substrate

All inter-agent communication is encapsulated as `uav_msgs/msg/TrafficMessage` and routed hop-by-hop through the FANET bus. The complete `TrafficMessage` schema (addressing, hop accounting, reliability/ACK fields, timestamps, and payload conventions) is documented in Appendix A.1. Here we focus on its system-level role: all telemetry and coordination traffic, including charging control, is carried over the same FANET substrate.

A key thesis design principle is that *charging requests and decisions traverse the same FANET substrate as telemetry*. Specifically, `CHARGE_REQUEST` and `CHARGE_DECISION` are encoded as `TrafficMessage` instances (`flow_type=1`) and routed via the same two-stage bus and fault-injector pipeline as DATA packets. This ensures that network degradation (weather-induced drops, routing failures, partitions) affects the charging control channel and the telemetry channel simultaneously, enabling a realistic evaluation of the interaction between network quality and charging protocol performance (RQ3).

Each message carries a globally unique `msg_id`, a `run_id`, and a `creation_time` timestamp, which together allow the logging pipeline to reconstruct per-packet causality.

3.4.2 Link and Impairment Model

Physical-layer connectivity is abstracted as follows. A link between two nodes is considered available if (a) their most recent `/fanet/status` beacons are both fresh (within `status_timeout_sec`), and (b) their Euclidean distance is within `min(comm_radius_uav, comm_radius_neighbor)`. This binary reachability check is performed by both the routing manager (for route construction) and UAV nodes (for neighbor table maintenance).

The fault injector applies a weather-dependent probabilistic drop to packets transiting from `/fanet/network_bus_raw` to `/fanet/network_bus`. The full drop model and parameter definitions are provided in Appendix B.2.

3.4.3 Routing Model

Routing is *control-plane centralised, data-plane distributed*.

The `routing_manager_node` maintains the global routing state:

1. It reads `/fanet/status` beacons from all nodes and builds a CH-only backbone graph, treating any node with `role=1` as an eligible backbone element. Nodes whose beacons have gone stale (beyond `status_timeout_sec`) are excluded.
2. A hysteresis margin (`hysteresis_margin_m`) prevents edges from being added and removed rapidly as nodes move near the range boundary.
3. Dijkstra shortest-path routing is run over the backbone graph. Each non-CH endpoint is assigned to the geographically nearest in-range CH (its *gateway*

CH). While Dijkstra’s algorithm is not optimal for highly dynamic topologies such as FANETs - where reactive protocols like AODV or OLSR are generally preferred - its use is justified in this context by the small, fixed fleet size and the periodic topology-refresh mechanism driven by status beacons. Under these conditions, the graph remains sparse and recomputation is negligible, making Dijkstra’s algorithm a sufficiently accurate and analytically tractable choice for the purposes of this study.

4. Per-node routing tables (`uav_msgs/msg/RoutingTable`) are published on `/fanet/routing_table`. Each table lists destination IDs and corresponding next-hop IDs; an empty next-hop string signifies unreachable.

Route recomputation is triggered periodically every `recompute_period_sec` and also on demand when a node publishes a routing event on `/fanet/routing_event` (e.g., upon a routing failure).

Forwarding is executed distributedly by CH UAVs: upon receiving a message addressed to them as the next hop, a CH checks whether it is the final destination; if not, it looks up the next hop for `dst_id` in its cached routing table and re-publishes the message with the updated `next_hop_id`. Loop prevention uses the `recent_hops` field; TTL acts as a backstop.

Unreachability is reported back as a DROP control message and is also surfaced by the routing manager on `/routing_manager/alerts` (e.g., `SINK_UNREACHABLE`, `UGV_UNREACHABLE`), which triggers the recovery subsystem.

3.4.4 Network Monitoring Hooks

The `network_monitor_node` subscribes to all three FANET topics (`/fanet/network_bus_raw`, `/fanet/network_bus`, `/fanet/delivered`) as an omniscient observer. For each message it maintains an in-memory `MsgRecord` keyed by `msg_id`. Online measurements derived from these records include:

- Per-category packet delivery ratio (PDR), computed as *delivered/generated* for each (`flow_type`, `control_type`) combination.
- Sliding-window PDR, mean delay, p95 delay, and jitter (window width `network_stats_window_sec`, default 10 s), published in real time on `/network_monitor/stats`.
- Specific control-channel PDR for `CHARGE_REQUEST` and `CHARGE_DECISION` messages, enabling direct measurement of the charging control channel reliability.

These quantities are written to `network_timeseries.csv` and `qos_metrics.csv` for offline analysis (see Section 3.9).

3.5 UAV Agent Design

3.5.1 UAV State Machine

Each UAV agent runs as a single `uav_node` instance regardless of its role. The charging behaviour is governed by a four-state finite state machine exported via `UavStatus.charging_state`:

ACTIVE (0) Normal operation: patrol task points, generate telemetry, and forward traffic (CH only).

GOING_TO_UGV (1) The UAV has received an accepted **CHARGE_DECISION** and is travelling toward the UGV docking region. Mobility priority shifts to the charge target pose derived from the decision payload.

CHARGING (2) The UAV has reached the UGV and is docked. Battery level increases at the configured charge rate. The UAV does not generate telemetry or forward packets during this state.

RETURNING (3) Charging has ended (battery target reached, or preemption command received) and the UAV is travelling back to its cluster operational region. On arrival, the state resets to **ACTIVE**.

The state transitions and their guards are:

- **ACTIVE** → **GOING_TO_UGV**: triggered when battery falls below `uav.battery_threshold` and an accepted **CHARGE_DECISION** is received.
- **GOING_TO_UGV** → **CHARGING**: triggered when the UAV arrives within the docking radius of the UGV.
- **CHARGING** → **RETURNING**: triggered on charge completion or on receipt of a **CHARGE_DECISION(reason=PREEMPTED)** message.
- **RETURNING** → **ACTIVE**: triggered on return to the cluster operational region.

An emergency mobility override allows a UAV to leave toward the Sink even when the normal deployment gates are still active, ensuring that energy depletion can be addressed regardless of mission phase.

3.5.2 Energy Dynamics in UAV Behaviour

Battery state is maintained internally as a continuous energy level (Wh) and exported as a percentage in `UavStatus.battery_level`. At each mobility tick the energy level is decremented by the current consumption rate, which is the sum of a base propulsion rate and a weather-dependent additive term derived from wind speed and rain intensity.

CH UAVs incur a relay overhead term in their energy model to reflect the additional computation and radio activity associated with multi-hop forwarding. Both the base drain and the CH overhead can be configured through YAML parameters. When the battery level reaches zero, the UAV publishes a **FailureEvent(failure_type=1)** and ceases all activity. Rather than permanent removal, the simulator implements a respawn mechanism: as battery approaches a critical threshold, the UAV returns to the sink and waits there until depletion. Once the **FailureEvent** is emitted, `respawnAtSink()` is invoked after a short delay (200 ms), restoring the UAV to full battery at the sink position with all state reset, after which it rejoins the fleet via a **RESPAWN_COMPLETED** control message.

3.5.3 Charging Request Generation

A UAV initiates a charging request when its battery energy falls at or below a dynamically computed `request_threshold`:

$$\text{request_threshold} = \begin{cases} E_{\text{to_ugv}} + E_{\text{reserve}} + E_{\text{buffer}} + E_{\text{adaptive_offset}}, & \text{if UGV position is known} \\ E_{\text{capacity}} \cdot P_{\text{battery_threshold}} + E_{\text{adaptive_offset}}, & \text{otherwise} \end{cases} \quad (3.1)$$

where $E_{\text{to_ugv}}$ is the estimated flight energy to reach the UGV, E_{reserve} and E_{buffer} are configurable safety margins (default 10 Wh each), $P_{\text{battery_threshold}}$ is a fallback percentage used when the UGV position is unknown, and $E_{\text{adaptive_offset}}$ is a small per-UAV staggering term derived from the UAV identifier to desynchronise simultaneous charge requests across the fleet.

When triggered, the UAV calls `requestCharge()`, which a

`TrafficMessage(flow_type=1, control_type="CHARGE_REQUEST")` addressed to `ugv_id` and routes it via FANET. For Members, the first hop is always the associated CH (`my_ch_id`); for CHs, the routing table is consulted directly.

The request message carries the current battery level and role in its metadata, which the UGV uses for scheduling. A retry timer (`chargeRequestRetryTick`) retransmits the request if no decision is received within

`uav.charge_decision_timeout_sec`, preventing indefinite waits.

An out-of-band mirror of the request is also published on

`/uav_fleet/charge_requests` as a `ChargeRequest` message to support network monitor logging.

3.6 UGV Charger and Scheduling Policies

3.6.1 UGV Charger Model

The UGV charger (`ugv_charger_node`) maintains:

- A *waiting queue* of incoming charging requests, each keyed by the requesting UAV's ID and associated with its battery level, role, and request timestamp.
- An *active session set* of UAVs currently docked and charging, bounded by `ugv.max_parallel_spots`.

The scheduler loop runs every 500 ms (`schedulerLoop` timer). At each invocation it evaluates the queue according to the selected policy, assigns available dock slots, and sends `CHARGE_DECISION` messages back to requesting UAVs via FANET.

The UGV also publishes status beacons on `/fanet/status` (enabling the routing manager to include it in next-hop computations) and a JSON charging snapshot on `/ugv/charging_snapshot` every second, which the network monitor uses to validate its own queue-length inference.

3.6.2 Scheduling Policy Definitions

Seven scheduling policies are implemented, selected by the `ugv.charging_policy` YAML parameter.

Non-preemptive policies.

fcfs — **First-Come First-Served**. Requests are served in arrival order with no role or urgency weighting.

edf — **Earliest-Deadline First**. Requests are ranked by estimated time-to-empty TTE_i , defined as

$$TTE_i = \frac{E_{\text{rem},i}}{C_i} \quad (3.2)$$

where $E_{\text{rem},i}$ is the remaining battery energy of UAV i and C_i is its current energy consumption rate. The UAV closest to depletion is served first.

role_priority — **Role-Priority**. CH UAVs (**role=1**) are always served before member UAVs (**role=0**); ties within a role class fall back to FCFS.

dynamic — **Dynamic Score**. Each queued request is ranked by a composite priority score

$$S_i = (\alpha \cdot R_i) + \left(\beta \cdot \frac{1}{TTE_i} \right) + (\gamma \cdot W_i) \quad (3.3)$$

where R_i is a binary role weight (1.5 for CH, 1.0 for member), W_i is the time the request has spent waiting in the queue, and α, β, γ are configurable scaling coefficients set in the run YAML. The wait term $\gamma \cdot W_i$ grows over time, preventing starvation of low-priority requests.

Preemptive variants. The preemptive variants of **p_edf**, **p_role_priority**, **p_dynamic_score** policies additionally allow interruption of an active session when both guards in the following equation are satisfied:

$$(S_j - S_i) > \Delta P_{\min} \quad \text{AND} \quad t_{\text{active},i} > t_{\min_charge} \quad (3.4)$$

where $\Delta P_{\min}$ is the minimum priority gap required to justify a swap, and $t_{\min_charge}$ is a minimum dwell time ensuring the interrupted UAV has gained at least some energy before being displaced. When preemption fires, the UGV sends a

CHARGE_DECISION(reason=PREEMPTED, target_action=STOP_CHARGING) to the victim through the FANET and re-queues it for future service.

3.6.3 Decision Outputs and Protocol Semantics

The UGV sends scheduling decisions as

TrafficMessage(flow_type=1, control_type="CHARGE_DECISION") routed through FANET to the requesting UAV. For readability and reproducibility, the complete payload field specification is provided in Appendix A.2.

The network monitor parses these fields from delivered **CHARGE_DECISION** messages and stores them in the **ChargeRecord** per-request in-memory structure, enabling policy-level attribution of every charging outcome in the offline analysis.

A critical design property is that *decisions must be deliverable over FANET*: the UGV uses the routing table to resolve the next hop toward the requesting UAV before transmitting. If the UAV is unreachable, the UGV drops the decision and emits a **DROP** message with reason **UNREACHABLE_CHARGE_DECISION_NEXT_HOP**, which is logged by the monitor. This means that network partitions or routing failures can cause decisions to be lost, making them subject to the same delivery uncertainty as telemetry.

3.7 Status, Heartbeat, Failure Detection, and Recovery

3.7.1 Status Reporting Model

Each active node in the system — every UAV, the UGV, and the sink — publishes periodic `UavStatus` beacons on `/fanet/status` at 1 Hz. The beacon carries the node's current pose, battery level, charging state, role, backbone-active flag, energy consumption rate, and a freshness timestamp.

These beacons serve three concurrent purposes: (i) input to the routing manager for backbone graph construction, (ii) input to the recovery manager for failure detection, and (iii) source data for the network monitor's `status_timeseries.csv`.

3.7.2 Heartbeat and Liveness Detection

UAV nodes publish `HEARTBEAT` control messages on the FANET bus every 1 second. The recovery manager tracks the freshness of both status beacons and heartbeats per node:

- A CH is declared *status-timed-out* if its most recent `/fanet/status` beacon is older than `recovery.status_timeout_sec`.
- A CH is declared *heartbeat-timed-out* if its most recent `HEARTBEAT` message observed on `/fanet/network_bus` is older than `recovery.heartbeat_timeout_sec`.
- A CH is declared *dead* immediately upon receipt of a `FailureEvent(failure_type=1)` from `/uav_fleet/failure_events`.

Routing-derived reachability alerts (`SINK_UNREACHABLE`, `UGV_UNREACHABLE`) published by the routing manager on `/routing_manager/alerts` also feed into the recovery manager, triggering recovery even when individual CHs remain alive but global connectivity is broken.

3.7.3 Recovery Workflows

The recovery manager (`recovery_manager_node`) operates as a centralised watchdog. Its internal state is controlled by boolean flags and a cooldown timestamp; a `watchdog` timer fires every 500 ms to check liveness conditions and invoke recovery when warranted.

A recovery epoch proceeds as follows:

1. `RECOVERY_START` is broadcast (`dst_id="broadcast"`) to all reachable nodes with the current epoch number.
2. The alive CH set is computed from the most recent status snapshot. If no CHs are alive, `MEMBER_FALLBACK` messages are sent to all Members with a target pose (UGV or sink).
3. If at least one CH is alive, a leader is elected by scoring each candidate as $\text{score} = \text{backbone_degree} \times \text{battery_percent}$. Members are reassigned to their nearest surviving CH via `CLUSTER_REASSIGN` control messages.
4. Task points are redistributed via `TASK_ASSIGN`. CHs may be redeployed to improve coverage or connectivity via `NEW_DEPLOYMENT`.
5. `RECOVERY_DONE` is broadcast to close the epoch.

The recovery manager maintains a `pending_acks` map and retransmits until acknowledged or until `max_ack_retries` is exceeded. A recovery cooldown

(`recovery_cooldown_sec`) prevents rapid repeated recoveries triggered by transient flapping.

Upon receiving a `CLUSTER_REASSIGN`, a UAV node updates its `my_ch_id` field, which governs its first-hop selection for all subsequent traffic. The cluster membership publisher (`ch_manager_node`) updates its published `ClusterInfo` snapshot accordingly.

Route tables are rebuilt implicitly on the next routing manager recomputation cycle following the topology change; no explicit route-flush message is sent.

3.8 ROS 2 Implementation View

3.8.1 Package and Node Organisation

The simulator is implemented as a modular ROS 2 workspace with separate packages for UAV agents, the UGV charging subsystem, routing, fault injection, recovery, and experiment logging. For completeness, the full package list and their responsibilities are provided in Appendix A.3.

3.8.2 Node–Topic/Service/Action Interface Table

Nodes communicate through ROS 2 topics to exchange status beacons, routed FANET traffic, charging requests/decisions, and recovery control messages. The complete node–interface mapping (publish/subscribe relationships) is provided in Appendix A.3.

3.8.3 Execution Model and Configuration

Each node uses the default ROS 2 single-threaded executor. Periodic behaviours are implemented as timer callbacks; reactive behaviours (forwarding, charging decisions, recovery actions) are implemented as subscription callbacks. The single-threaded model avoids concurrency hazards but means that a long-running callback (e.g., routing recomputation on a large graph) can delay other callbacks within the same node.

The entire experiment is orchestrated by `system_bringup/launch/experiment.launch.py`, which:

1. Reads the scenario YAML
(e.g., `system_bringup/config/runs/ugv_edf.yaml`).
2. Instantiates one `uav_node` per UAV defined in the YAML, naming each `<uav_id>_<run_id>`.
3. Instantiates one `ch_manager_node` per cluster.
4. Instantiates all supporting nodes (weather, fault injector, routing manager, coverage planner, sink, UGV, recovery manager, network monitor).
5. Attaches a `Shutdown` event to a timeout timer, enforcing the `global.experiment_timeout_s` wall-clock run duration.

The YAML run configuration is the single source of truth for all node parameters, ID strings, seeds, and policy selection. Changing `ugv.charging_policy` between YAML files while keeping all other parameters identical is the canonical approach for controlled policy comparisons.

3.9 Experiment Configuration and Data Logging Design

3.9.1 Run Configuration and Reproducibility

Each experiment run is fully specified by:

- A YAML run file under `system_bringup/config/runs/`. The file specifies UAV IDs, cluster assignments, scenario bounds, `ugv.charging_policy`, weather mode, seeds, and experiment duration.
- A `run_id` string passed on the command line, used as the output subdirectory name and embedded in every logged row.
- A `global.rng_seed` fixing the master RNG, and a task-point seed fixing the task layout.

In the baseline simulator configuration used throughout this thesis, the aerial network consists of two cluster heads, three member UAVs, one sink UAV, and one UGV acting as the charging and scheduling platform.

Output artefacts are written to `<output_dir>/<run_id>/`. All CSV files are opened in append mode with a header written only if the file does not yet exist, so that a process restart with the same `run_id` appends new rows rather than overwriting previous data. A `run_instance_id` (wall-clock nanoseconds at process start) is included in every row, allowing rows from different process invocations sharing the same `run_id` to be distinguished.

3.9.2 Logged Data Products

The network monitor produces two complementary families of artefacts: (i) append-only event logs (packets, charging requests/decisions and sessions, failures, and recovery actions), and (ii) time-series logs sampled at 1 Hz (status, weather, queue dynamics, and network QoS). This separation supports post-hoc reconstruction of causal chains (per-message and per-request) while also enabling trend analysis over time.

The complete logging schema (all files and their fields) is provided in Appendix [D.1](#).

3.9.3 Metric Extraction Pipeline Preview

Chapter [4](#) derives experimental metrics such as packet-level reliability and delay by joining generated, delivered, and drop events, while charging performance is computed from request/decision/session events and role annotations. Recovery behaviour are derived from death and recovery event traces.

For details see the post-processing pipeline provided in Appendix [D.2](#).

Chapter 4

Experiment & Results

4.1 Experimental Design and Evaluation Methodology

To keep the main text focused on interpretation, formal metric definitions and post-processing rules are reported in Appendix E.1, Appendix E.2, and Appendix E.3.

4.1.1 Experimental Objective and Research-Question Mapping

The purpose of the experiments is to determine how different UAV charging scheduling policies affect system performance when charging coordination is carried over the same FANET used for telemetry. Since all other scenario conditions are held constant, the comparison isolates the effect of the scheduling policy on both resource allocation and network behaviour.

The evaluation is organised around the three research questions introduced in Chapter 1:

- **RQ1 – FANET robustness and routing quality.** The first objective is to assess whether different charging outcomes are associated with different levels of telemetry reliability, measured primarily through packet delivery ratio (PDR), end-to-end delay, and their temporal evolution.
- **RQ2 – Charging policy, mission continuity, and survivability.** The second objective is to compare the ability of each policy to preserve UAV availability through successful charging, reduced waiting burden, and lower battery depletion rates.
- **RQ3 – Shared communication substrate and cross-layer coupling.** The third objective is to examine whether degradation in the FANET also weakens the charging-control loop itself, thereby creating a feedback mechanism in which poor connectivity leads to failed charging, further depletion, and additional network degradation.

4.1.2 Protocols Under Comparison

Seven charging policies are evaluated in Test Round 2. They include both non-preemptive and preemptive strategies:

- `ugv_fcfs`: non-preemptive first-come, first-served;
- `ugv_edf`: non-preemptive earliest deadline first;

- `ugv_role_priority`: non-preemptive role-priority scheduling;
- `ugv_dynamic`: non-preemptive dynamic-score scheduling;
- `ugv_p_edf`: preemptive earliest deadline first;
- `ugv_p_role_priority`: preemptive role-priority scheduling;
- `ugv_p_dynamic_score`: preemptive dynamic-score scheduling.

The non-preemptive policies decide only which waiting UAV should be served next when a dock becomes available. By contrast, the preemptive policies are allowed to interrupt a lower-priority charging allocation when a more urgent request arrives.

4.1.3 Replication Strategy, Dataset Scope, and Artifact Availability

Each of the seven protocols was executed in three independent replicate runs, yielding a total of 21 runs for Test Round 2. All runs span the same simulated mission duration of 180 minutes and were produced under the same scenario configuration apart from the policy switch. This balanced design enables both within-policy stability checks and cross-policy comparisons under matched experimental conditions.

In the overall experimental workflow, **Test Round 1** was used primarily as a preliminary validation stage to assess simulator stability, inspect logging behaviour, and identify implementation issues before conducting the final comparative study. The results presented in this chapter are instead based on **Test Round 2**, which is the main evaluation dataset used for protocol comparison.

A further important difference is that the weather node in Test Round 2 was configured in **Markov mode**, so that weather regimes evolve over time through macrostate transitions rather than remaining fixed throughout the entire run. This choice makes the evaluation more representative of time-varying environmental stress and allows the charging and network subsystems to be tested under changing conditions.

The analysis therefore adopts the protocol as the primary comparison factor and the replicate run as the unit of repeated observation. Aggregate protocol summaries are reported from the set of three runs belonging to the same policy, while temporal plots preserve run-level trajectories in order to expose instability, outliers, and failure cascades that would be hidden by averaging alone. The chapter concludes with a controlled follow-up validation under fixed sunny weather, designed to test whether the main survivability-related pattern observed in Test Round 2 remains visible when weather variance is reduced.

For reproducibility, the experiment outputs and the plotting/analysis scripts used to generate the figures discussed in this chapter are maintained in the project GitHub repository.¹ In particular, the Test Round 2 dataset is organised under `experiment_data_collection/test_round2`, while the corresponding figures are stored under `analysis/test_round2/figures/`.

4.1.4 Evaluation Metrics

The evaluation uses four families of metrics, each corresponding to one aspect of the system behaviour under study.

¹https://github.com/kk-Syuer/UAV_UGV_monitoring

Network-quality metrics. The primary network indicator is the packet delivery ratio (PDR), used as the main measure of telemetry reliability. End-to-end delay is also reported, but interpreted with caution because later sections show that raw delay values may be biased by delivery conditioning when the network collapses and only a restricted subset of packets continues to arrive. Window-level PDR and delay are further used to study temporal degradation and recovery.

Charging-performance metrics. Charging effectiveness is evaluated using charging success rate, timeout rate, ROUTING_DROP rate, decision latency, effective wait, per-session recovered energy, and dock utilisation. These metrics distinguish between failures due to queue pressure and failures due to the inability of the request to reach the scheduler.

Fleet-survivability metrics. Mission continuity is assessed through the number and temporal distribution of battery depletion events. The analysis also distinguishes between depletions affecting cluster heads and member UAVs, since the loss of a cluster head carries a larger topological consequence for the FANET backbone.

Cross-layer metrics. To support the main argument of the chapter, network and charging indicators are not analysed in isolation. Instead, the results later examine statistical associations between PDR and selected charging-related variables, including charge success rate, ROUTING_DROP rate, dock utilisation, queue length, and depletion burden. The purpose is not to claim strict causality from correlation alone, but to test whether the observed protocol differences are consistent with the proposed mechanism linking charging coordination to network quality. Formal definitions of these metrics are provided in Appendix E.1.

4.1.5 Dataset Reliability Note

Before comparing protocol outcomes, the Test Round 2 dataset was checked for completeness and structural consistency. All seven protocols were available with three replicate runs each, the required analysis files were present for all runs, and all runs reached the target mission duration. Additional preprocessing notes, including sentinel-value handling, duplicated windows, and record-filtering rules, are reported in Appendix E.6.

4.2 Aggregate Protocol-Level Results

This section compares the seven charging policies at an aggregate level using the three replicate runs available for each protocol. The discussion focuses on three dimensions: overall network performance, charging effectiveness, and fleet survivability. Detailed metric definitions and aggregation rules are reported in Appendix E.1 and Appendix E.2.

4.2.1 Overall Network Performance

At the aggregate level, `ugv_p_edf` achieves the best overall network performance, with the highest mean packet delivery ratio among all tested policies. `ugv_fcfs` and `ugv_dynamic` follow, while `ugv_role_priority` and `ugv_p_role_priority` form the weakest tail of the ranking, as shown in Fig. 4.1. A second relevant observation is the difference in stability across replicates. Although the mean PDR range

across protocols is not extremely wide, some policies show much higher between-run variance than others. In particular, `ugv_edf` and `ugv_p_role_priority` display visibly larger dispersion, indicating that their behaviour is less robust under the same nominal conditions. This instability becomes more visible in the time-series overlays of Fig. 4.7 and becomes important later when analysing failure cascades and outlier runs. Delay is treated as a secondary indicator throughout; its interpretation is revisited in Section 4.5.

Overall, the aggregate network comparison already suggests that preemptive, urgency-aware scheduling is beneficial at system level. The advantage of `ugv_p_edf` is not limited to queue handling alone; rather, it appears associated with better preservation of the operational fleet and therefore with more reliable multi-hop communication.

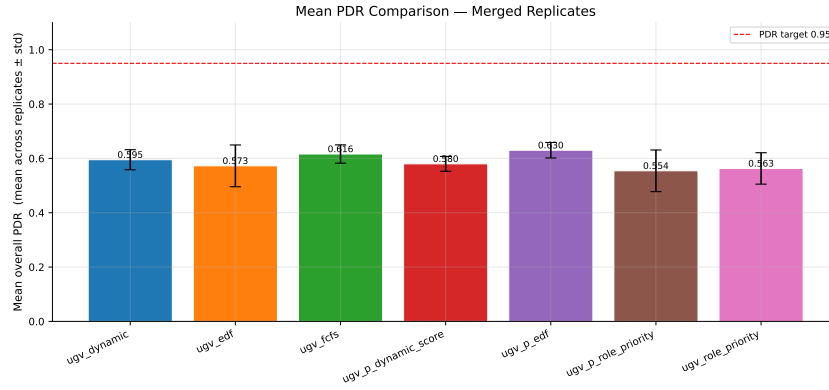


Figure 4.1. reports cumulative PDR from `qos_metrics.csv` (delivered / generated over the full run)

4.2.2 Charging Scheduler Performance

The charging results reinforce the network-level ranking. Among the seven protocols, `ugv_p_edf` achieves the highest charge success rate, indicating that preemptive deadline-based scheduling is the most effective at converting requests into successful dock starts under contention. `ugv_dynamic` is also strong, combining a high success rate with comparatively low decision latency and effective wait (Fig. 4.2).

At the opposite end, `ugv_role_priority` and `ugv_edf` show the lowest success rates. Their weaker performance is reflected not only in the number of successful charging outcomes, but also in the lower energy recovered per session and the higher prevalence of unresolved requests. This indicates that poor charging performance is not merely a matter of slower queue progression, but a more general difficulty in sustaining the service loop under adverse conditions.

An important result is that routing-related request loss remains a major failure mode across all protocols. In other words, a non-negligible share of charging failures occurs before the scheduler can even make a decision, as also shown by the outcome breakdown in Fig. 4.3. This supports the thesis claim that the charging subsystem is tightly coupled to network health: when the FANET is degraded, some UAVs are unable to reliably reach the UGV with their requests, and scheduling quality alone cannot compensate for that loss.

For this reason, the charging metrics should be interpreted jointly. A protocol with a moderate decision time but a high success rate is more valuable than one with

fast decisions on the small subset of requests that actually arrive. Later sections return to this distinction and show that request reachability is more consequential than raw decision latency.

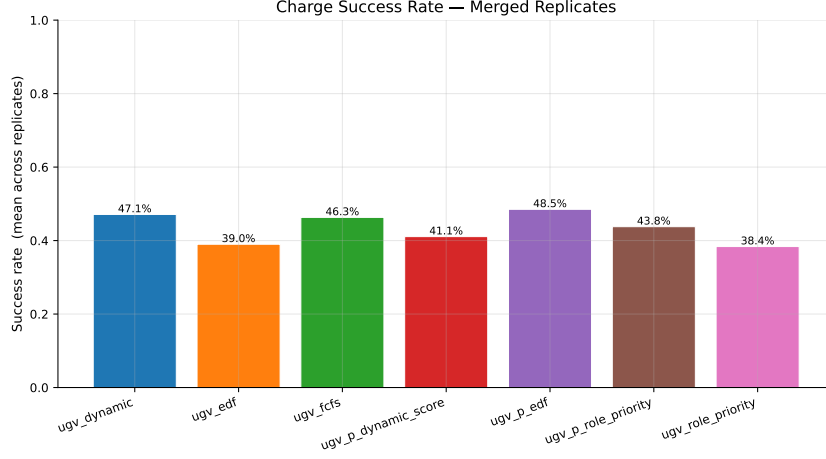


Figure 4.2. Charging success rate by protocol.

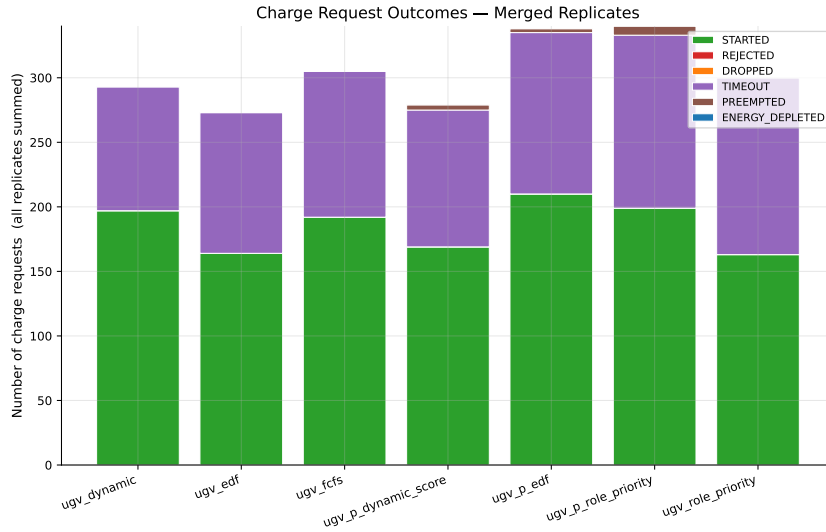


Figure 4.3. Breakdown of charging outcomes by protocol.

4.2.3 Fleet Survivability

Fleet survivability further clarifies the protocol differences. The lowest depletion burden is observed under `ugv_p_edf`, followed by `ugv_dynamic`, while `ugv_role_priority` and especially `ugv_edf` show markedly worse outcomes. This confirms that the most effective charging policies are also the ones that best preserve the availability of the aerial fleet over the full mission horizon, as shown in Fig. 4.4.

The depletion statistics are particularly important because UAV loss has a direct network consequence. When member UAVs deplete, sensing and forwarding capacity are reduced; when cluster heads deplete, the impact is even more severe because the

communication backbone itself becomes unstable. For this reason, depletion count is not only a survivability metric but also a mechanism-level indicator that helps explain subsequent differences in PDR.

The aggregate results also reveal a difference between structural weakness and outlier-driven instability. `ugv_role_priority` performs poorly in a relatively consistent way across runs, which suggests an intrinsic limitation of the policy. By contrast, `ugv_edf` is strongly affected by one extreme run, indicating that its average result is partly shaped by catastrophic instability rather than uniformly poor behaviour in every replicate.

Taken together, the aggregate evidence points to a coherent pattern: protocols that achieve more successful charging outcomes tend to show fewer depletions, and protocols with fewer depletions tend to preserve better network reliability.

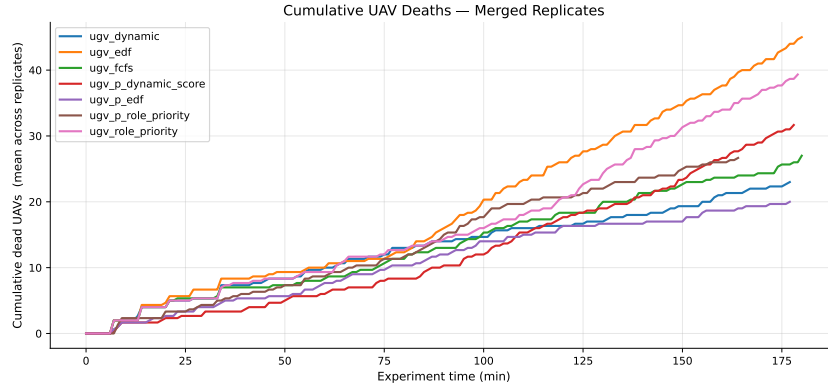


Figure 4.4. Cumulative UAV battery depletion events by protocol.

4.2.4 Aggregate Trade-Off View

When the metrics are read together, `ugv_p_edf` emerges as the strongest overall policy in Test Round 2. It combines the highest mean PDR, the highest charging success rate, and the lowest depletion burden, making it the most balanced solution among the evaluated protocols. `ugv_dynamic` is also competitive, especially in terms of scheduler responsiveness, but its overall network and survivability performance remains slightly weaker.

At the opposite end, `ugv_role_priority` is the weakest policy overall. Its lower charging success, higher timeout burden, and heavier depletion load are consistent with its weaker network outcomes. `ugv_p_role_priority` is also unfavourable, although for partly different reasons, since its poor network ranking coexists with a comparatively healthier surviving fleet and therefore requires more careful interpretation in the delay and cross-layer analysis.

These aggregate results establish the broad ranking of the policies, but they do not yet explain *how* degradation develops over time. For that reason, the next section moves from protocol-level averages to temporal trajectories of PDR, queue pressure, dock utilisation, and depletion events.

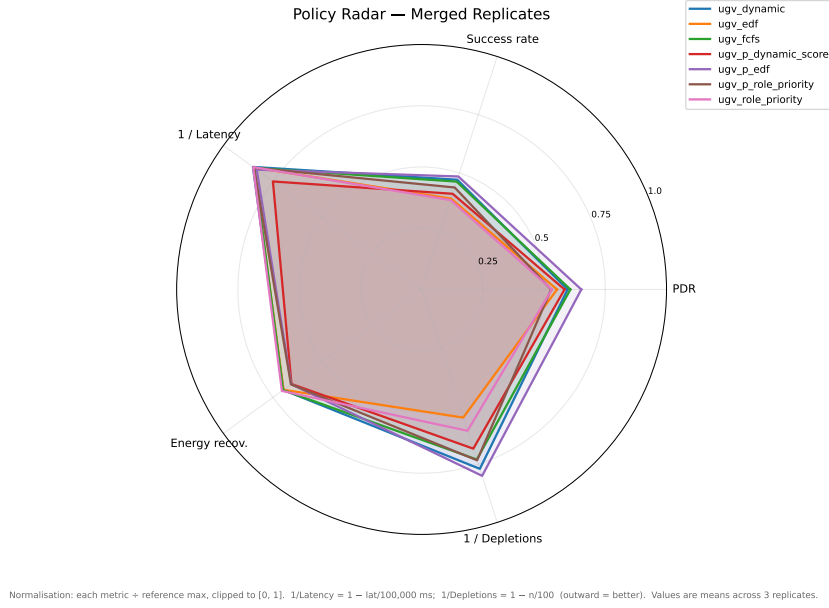


Figure 4.5. Aggregate trade-off across major protocol KPIs.

4.3 Temporal Dynamics of Charging and Network Quality

4.3.1 PDR Evolution Over Time

The time-series plots show that the protocols differ not only in average PDR, but also in the way reliability evolves over the 180-minute mission. In general, `ugv_p_edf` exhibits the most stable trajectory, with smaller and less persistent drops in delivery performance. By contrast, `ugv_edf` and `ugv_p_role_priority` show much larger between-run variability, including episodes of abrupt degradation that are not visible from the mean alone.

This temporal view is important because some protocols achieve similar aggregate PDR values while behaving very differently over time. A protocol with modest but stable reliability may be preferable to one with a similar mean but repeated collapse intervals. For this reason, the time-series plots in Fig. 4.6 and Fig. 4.7 are used throughout the chapter to separate structural weakness from outlier-driven behaviour. The dip-threshold rule used in the event-aligned plots is defined in Appendix E.4.

4.3.2 Queue Pressure, Dock Utilisation, and Charging Throughput

The charging time-series help clarify whether poor outcomes are mainly caused by persistent queue overload or by failures earlier in the control path. Across protocols, dock utilisation remains within a relatively narrow range compared with the much larger variation observed in timeout rates and routing-drop rates. This suggests that unsuccessful charging is not explained solely by full docks or long local queues.

A more informative pattern is the co-evolution of queue activity and charging throughput. In healthier runs, requests continue to arrive, docks remain active, and cumulative charged energy grows steadily. In weaker runs, charging throughput flattens over time and dock utilisation may fall rather than rise, indicating that the

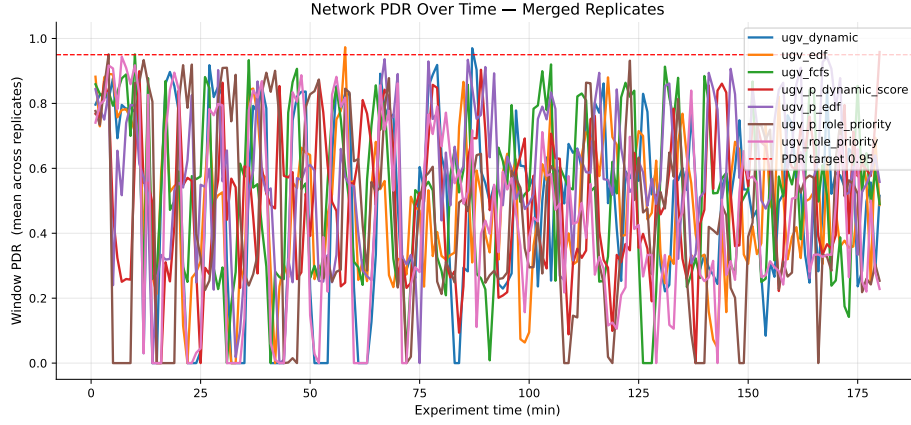


Figure 4.6. Merged PDR over time across protocols.

system is no longer sustaining a healthy flow of successful requests. In this sense, low utilisation is not necessarily a sign of low demand; it can also be a symptom of network degradation and fleet attrition.

This distinction is useful for interpreting timeout behaviour. If timeout burden grows without a corresponding sustained increase in dock pressure, the cause is more likely to lie in policy decisions or request reachability than in simple charger saturation. These patterns are visible in Fig. 4.8, Fig. 4.9, and Fig. 4.10.

4.3.3 Weather Context and Time-Varying Degradation

Because Test Round 2 uses the weather node in Markov mode without fixed weather seed, network conditions are subject to time-varying environmental stress rather than a fixed regime for the entire experiment. The weather-conditioned plots therefore provide important context for interpreting temporal performance changes. In particular, lower PDR and higher delay are expected during harsher regimes, and these periods can coincide with higher charging disruption.

The value of these plots is to show that protocols are being evaluated under changing external conditions rather than a fully static environment. This makes the comparison more realistic and also helps explain why some degradation episodes appear clustered in time instead of being uniformly distributed across the mission horizon.

For this reason, temporal drops in PDR or charging throughput should be read together with the weather timeline in Fig. 4.13 and with the weather-conditioned comparisons in Fig. 4.11 and Fig. 4.12, especially before drawing conclusions about whether a failure is policy-driven, environment-driven, or the result of both factors acting together.

4.4 Cross-Layer Analysis: From Scheduling to Network Quality

The previous sections showed that the protocol ranking is broadly consistent across network, charging, and survivability metrics. This section makes that relationship explicit by analysing how charging outcomes are associated with FANET quality. The goal is not to claim causality from correlation alone, but to test whether

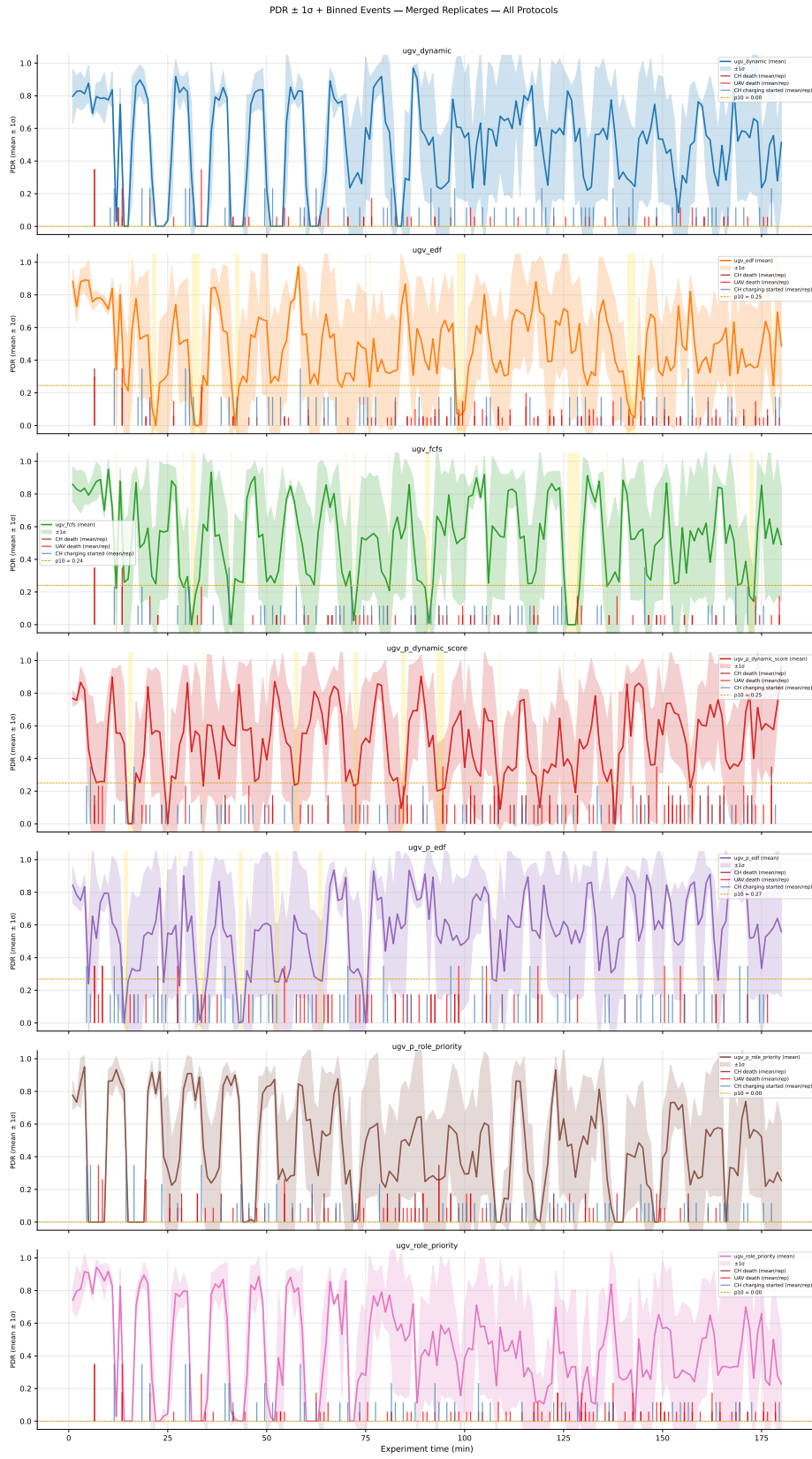


Figure 4.7. PDR evolution with replicate variance and event overlays. It reports the temporal mean of `window_pdr` samples from `network_timeseries.csv` (windowed 10-second sliding values, sentinel values excluded).

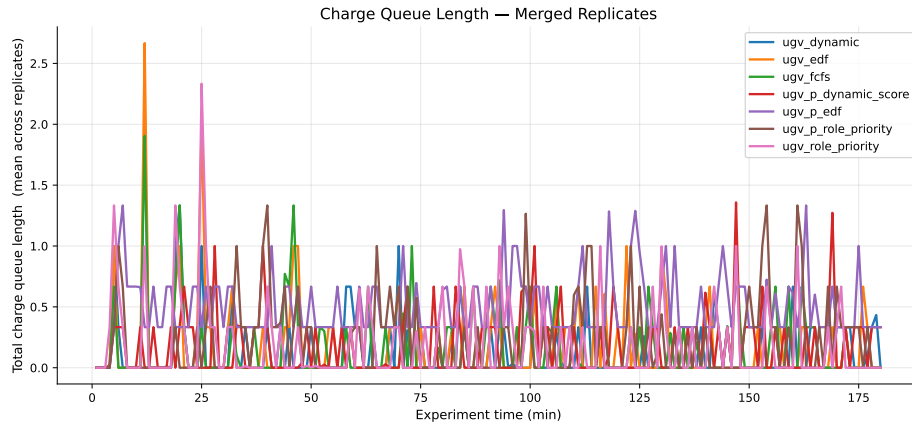


Figure 4.8. Queue length over time across protocols.

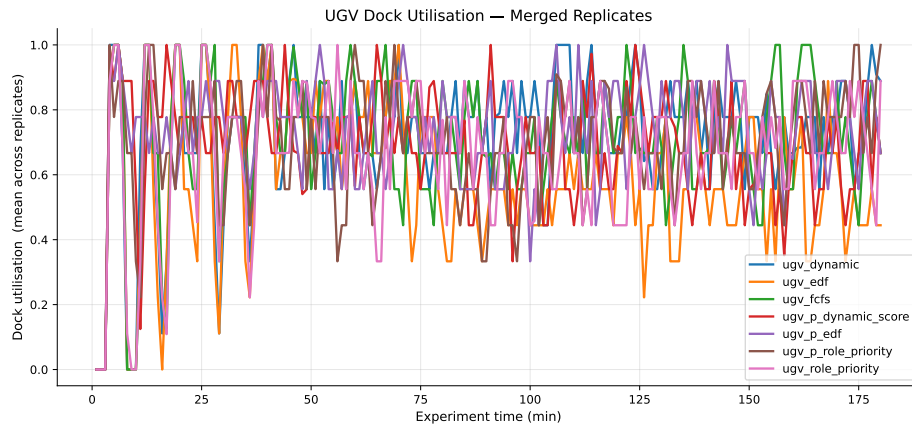


Figure 4.9. Dock utilisation over time across protocols.

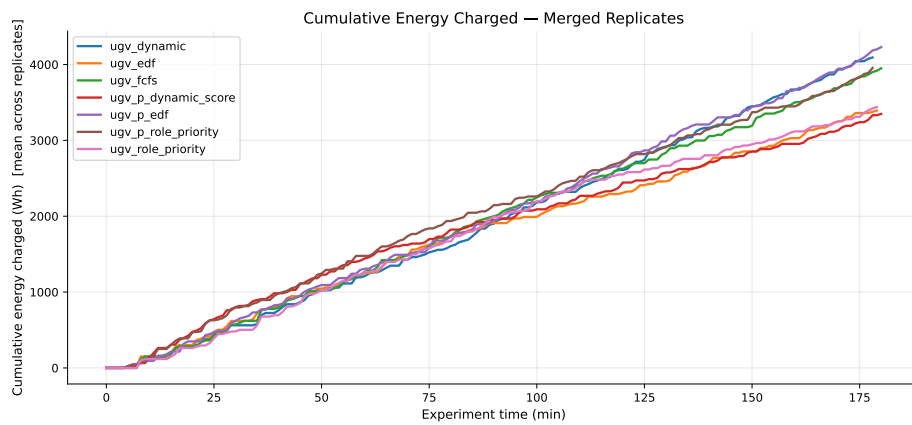


Figure 4.10. Cumulative charged energy over time across protocols.

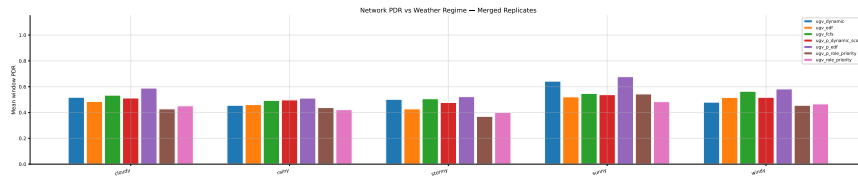


Figure 4.11. Mean PDR by protocol and weather regime.

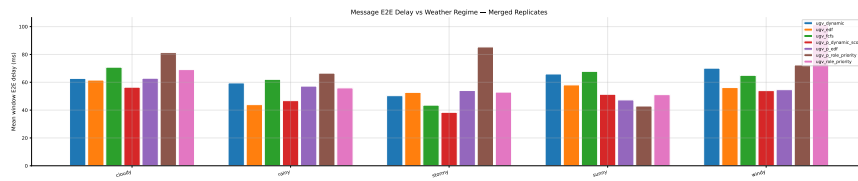


Figure 4.12. Mean delay by protocol and weather regime.

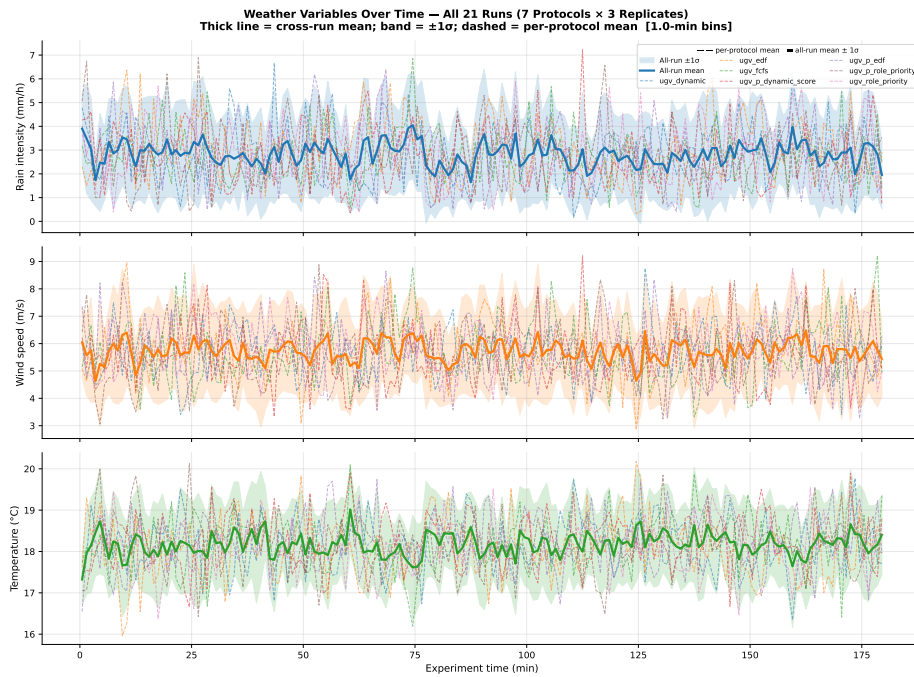


Figure 4.13. Weather regime evolution over time.

the observed protocol differences are consistent with the mechanism proposed in this thesis: better charging continuity preserves more UAVs, and a healthier fleet preserves more stable multi-hop communication.

4.4.1 Statistical Association Between Charging Outcomes and PDR

Across the 21 runs, charge success rate is positively associated with mean PDR, while charge-request `ROUTING_DROP` rate is negatively associated with it. In the round-2 analysis, charge success shows both positive Pearson and Spearman correlations with PDR, whereas `ROUTING_DROP` shows negative correlations of similar magnitude. This indicates that protocols which convert more requests into successful charging sessions also tend to maintain better telemetry delivery, as shown in Fig. 4.14 and Fig. 4.15.

A second important result is that battery depletion burden is strongly negatively associated with PDR. This makes depletion a useful mediating variable between the charging layer and the communication layer: poor charging outcomes are reflected in more frequent UAV loss, and heavier UAV loss is reflected in lower network reliability. In this sense, depletion is not only an outcome metric, but also a bridge that helps explain why charging policy differences reappear at network level.

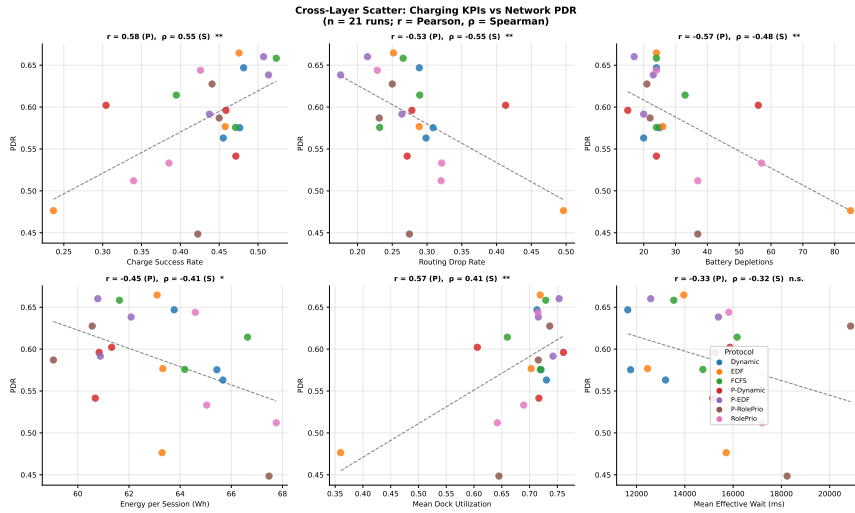


Figure 4.14. Association between charging success and mean PDR.

4.4.2 Mechanism Interpretation

The aggregate and temporal results support a coherent mechanism chain. First, the scheduling policy affects whether a charging request succeeds, times out, or fails before reaching the scheduler. Second, unsuccessful charging reduces the amount of energy effectively returned to the fleet. Third, reduced energy recovery increases the probability of UAV depletion. Finally, heavier depletion weakens the relay structure of the FANET, especially when cluster heads are affected, and this lowers packet delivery performance. This chain is summarised in Fig. 4.16 and its temporal signature is visible in Fig. 4.17.

This interpretation also explains why the strongest protocol in this study is `ugv_p_edf`. Its advantage is not limited to queue efficiency at the charger. Rather, it appears to preserve fleet continuity more effectively, and that continuity in turn

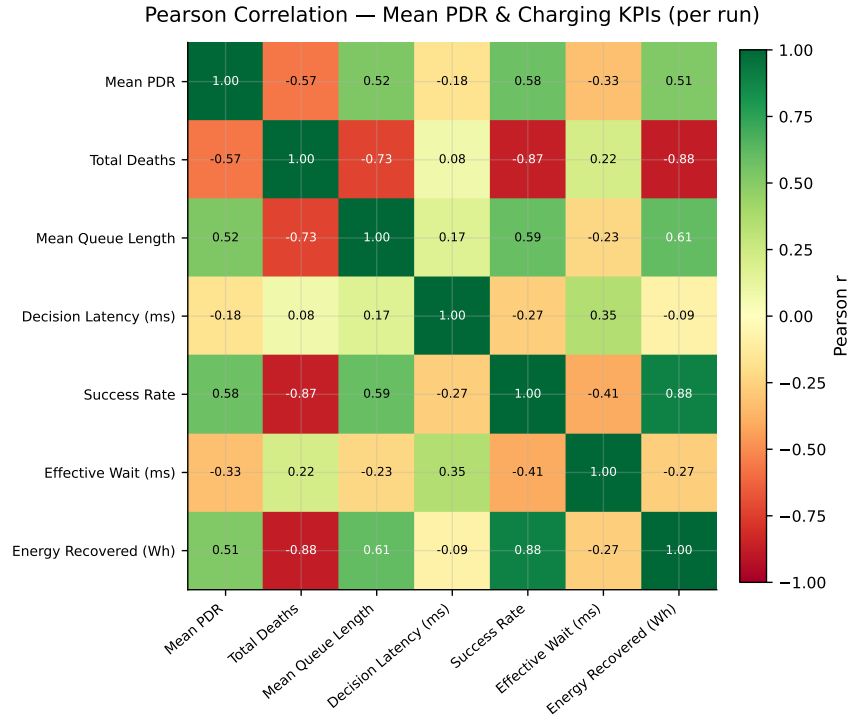


Figure 4.15. Correlation structure of PDR and selected charging indicators.

supports higher PDR. Conversely, the weakest protocols do not fail only because they make poor local scheduling decisions; they also allow a larger degradation loop to form between charging failure, UAV loss, and network deterioration.

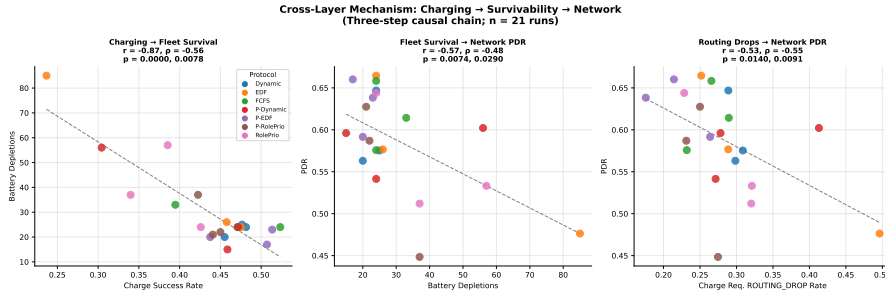


Figure 4.16. Cross-layer mechanism linking charging outcomes, depletion, and PDR.

4.4.3 Why Request Reachability Matters More Than Decision Speed

One of the clearest findings of the cross-layer analysis is that decision latency and effective waiting time are much weaker predictors of PDR than request reachability. The round-2 report finds no strong or significant association between mean PDR and raw decision latency, whereas ROUTING_DROP rate shows a clear negative association. In practical terms, this means that the dominant bottleneck is often not how quickly the scheduler responds, but whether the request reaches the scheduler at all.

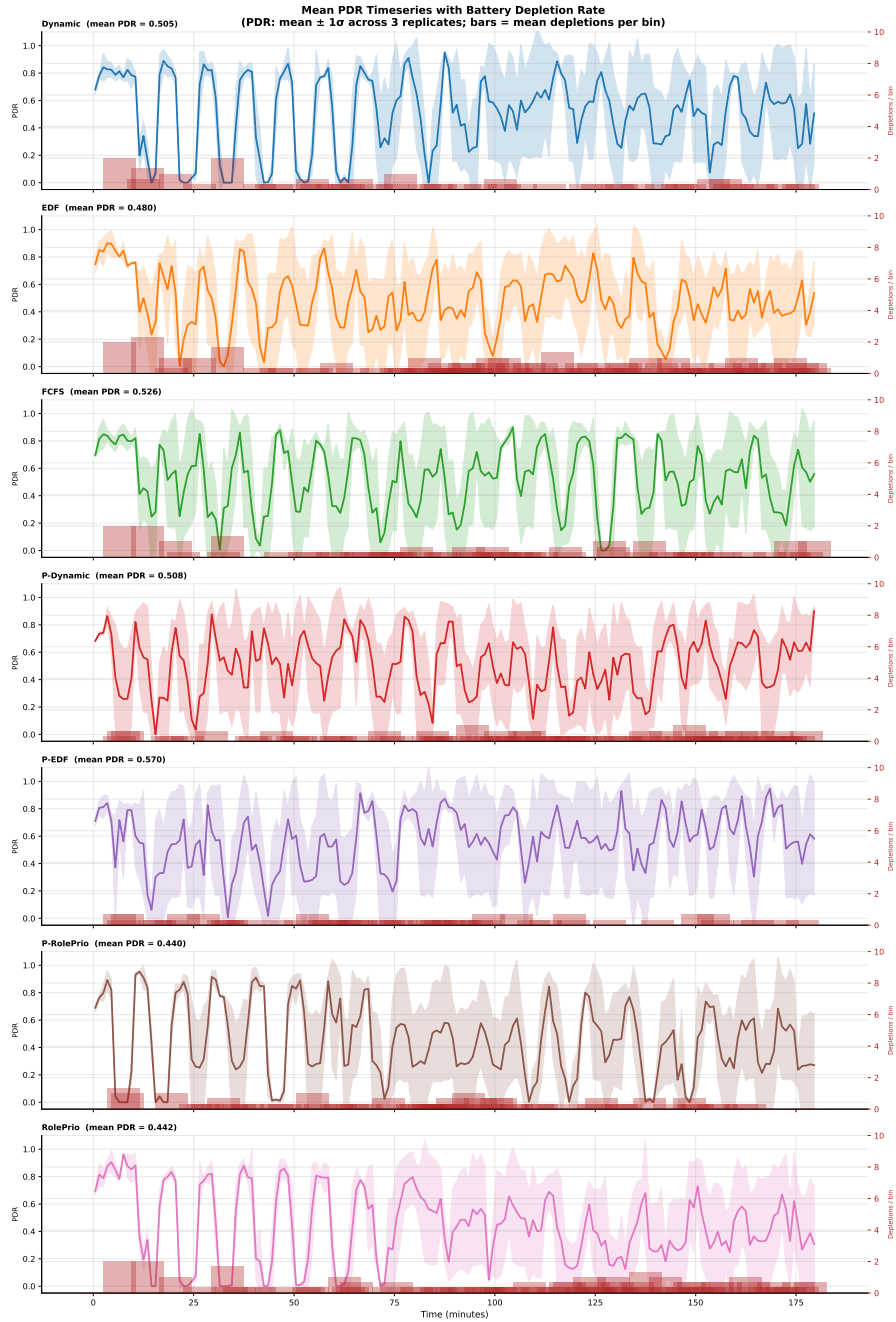


Figure 4.17. Temporal co-evolution of mean of `window_pdr` samples from `network_timeseries.csv` and depletion burden.

This distinction is important for the interpretation of protocol quality. A policy may appear computationally responsive once a request is available locally at the UGV, yet still perform poorly at system level if many requests are lost in transit during adverse network conditions. In the present architecture, charging coordination depends on the same FANET that carries telemetry, so network degradation can directly disrupt access to the charging service itself.

The result therefore strengthens one of the main thesis claims: in a shared-substrate design, charging performance cannot be evaluated only in terms of queue discipline or local scheduling speed. It must also be evaluated in terms of end-to-end control reachability under network stress, as shown in Fig. 4.18 and Fig. 4.19.

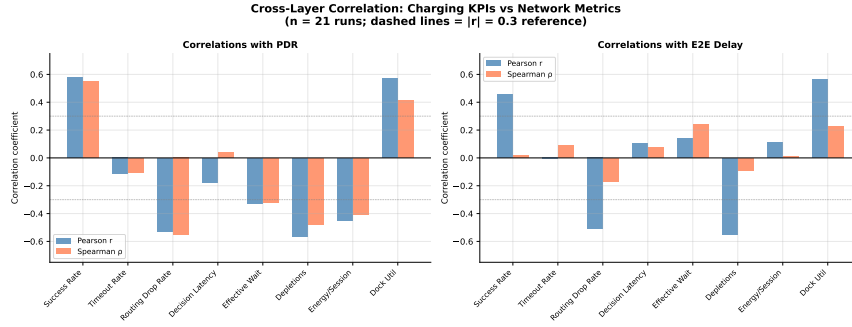


Figure 4.18. Correlation strength of selected charging indicators with PDR.

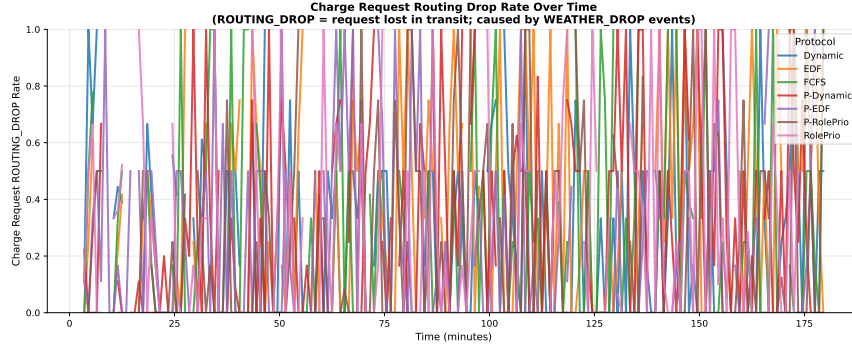


Figure 4.19. Routing-drop burden over time.

4.4.4 The CH Priority Paradox: Why Role-Based Protocols Underperform

The role-based protocols, `ugv_role_priority` and `ugv_p_role_priority`, perform worse than expected despite explicitly giving cluster heads (CHs) the highest scheduling priority. In the present simulator, this underperformance is not mainly caused by dock-level contention. Instead, it arises from a more structural effect: the system contains only two CHs, and a CH disables packet forwarding while charging. As a result, when both CHs are admitted to charging in close temporal proximity, the relay backbone becomes temporarily unavailable for the three member UAVs.

Hypothesis. The two CHs have identical battery capacity and broadly similar relay-intensive duty cycles, so they tend to approach the charging threshold at nearly

the same time. This produces repeated paired CH charge requests. Under role-based scheduling, both CH requests are promoted ahead of member requests and are likely to be served in close succession, or even in parallel given the multi-dock UGV configuration. Because a charging CH does not forward packets, simultaneous CH charging removes the only CH-level relays available to the members. The members then lose their path to the UGV scheduler, causing higher `ROUTING_DROP` and lower charging success. As member survivability deteriorates, the network weakens further and the failure loop amplifies.

Empirical evidence. The left panel of Figure 4.21 shows that the two CHs often generate charge requests within a short temporal gap across protocols, confirming that CH demand is naturally synchronized. This is consistent with the fleet design: the CHs have the same battery capacity and similar relay workload, so they deplete on correlated time scales.

Figure 4.20 shows that the role-based protocols are associated with worse member-side outcomes than the stronger urgency-based alternatives, including lower member charge success and heavier member routing-drop burden. This pattern is consistent with the loss of relay availability when CH charging events become temporally clustered. The protocol-level KPI overview in Figure 4.22 supports the same interpretation: despite prioritising CHs, the role-based policies still exhibit weaker PDR, poorer overall charge success, and heavier depletion burden.

The important point is that the paradox does not require CHs to wait behind each other at the dock. Since the UGV offers multiple docking spots, both CHs can be served quickly. However, this local scheduling success becomes a system-level problem because it removes both relay-capable CHs from the forwarding layer at the same time.

Interpretation and design implication. The main failure mode of role-based scheduling is therefore simultaneous CH withdrawal from the relay backbone rather than simple queue inefficiency. In a fleet with only two CHs, prioritising both of them too aggressively can momentarily improve CH access to charging while severely damaging communication access for the members.

A more suitable fix would be to retain role awareness but avoid concurrent CH charging whenever both CHs are the only active high-level relays. One possible design is a relay-preserving constraint: if one CH is already charging, the second CH may be delayed briefly or conditionally admitted only when member connectivity can still be preserved. This would reduce backbone collapse without abandoning the role hierarchy entirely.

Further implementation details for the role-stratified metrics, CH synchronization analysis, and relay-loss interpretation should be documented in the appendix (Appendix E.6.5, Appendix E.6.6).

4.5 Delay Interpretation and Measurement Bias

End-to-end delay must be interpreted alongside PDR, not independently. Delay is computed only over delivered packets; under heavy UAV attrition, many multi-hop transmissions are lost, leaving a surviving packet set that traverses a simpler, shorter topology. This produces a conditioning bias: protocols with worse survivability can appear to have lower delay precisely because their network has collapsed. The weak relationship between PDR and mean delay in Fig. 4.24 confirms that delay carries limited protocol-discriminating information in this dataset. For this reason, the

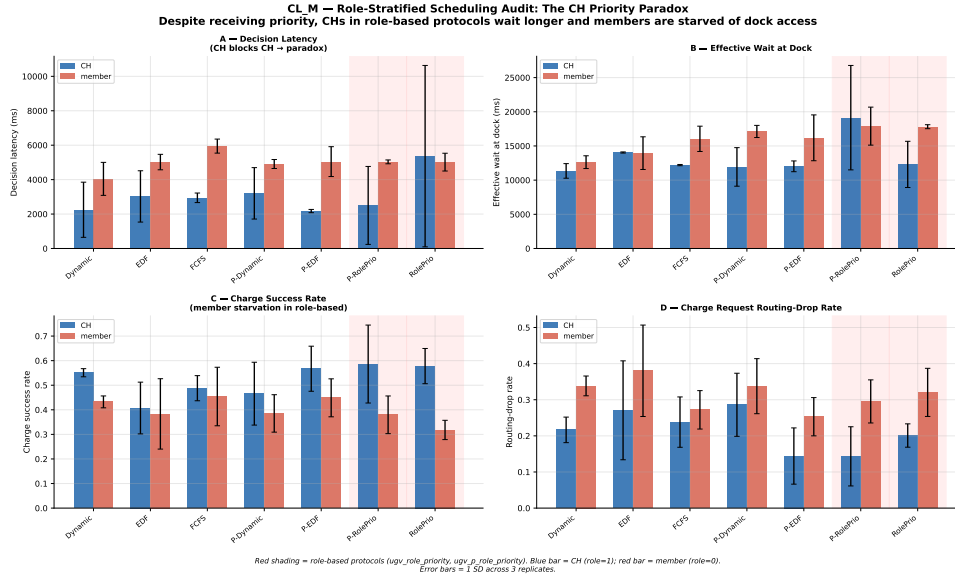


Figure 4.20. Role-stratified scheduling audit. In the role-based protocols, CHs do not receive genuinely effective priority, while members experience lower charging success and higher routing-drop burden.

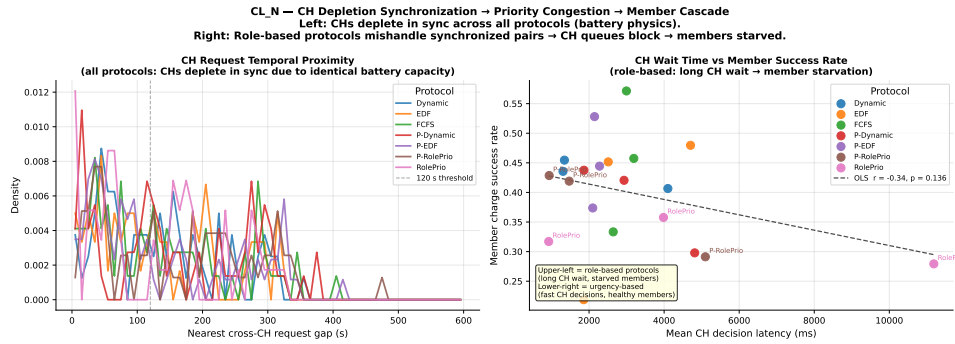


Figure 4.21. CH synchronization and member-starvation cascade. Left: CH charge requests occur in close temporal proximity across protocols. Right: runs with longer CH wait tend to exhibit weaker member charge success.

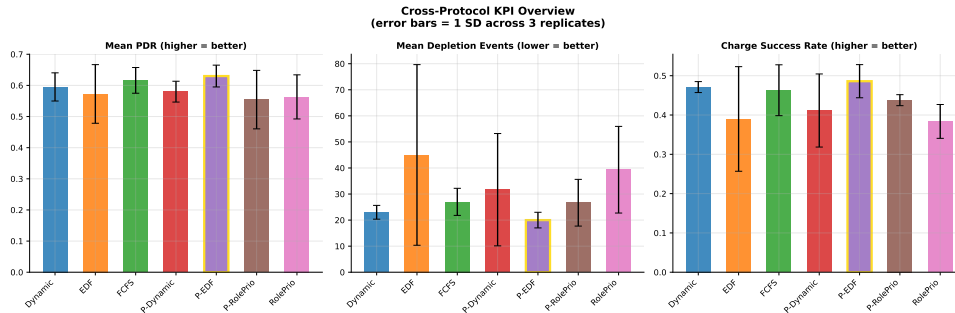


Figure 4.22. Cross-protocol KPI overview. The role-based protocols remain weak in overall PDR, depletion burden, and charge success despite explicitly prioritising cluster heads.

comparative evaluation is based on PDR, charging success, and depletion burden, while PDR-weighted delay is discussed only as a robustness check.

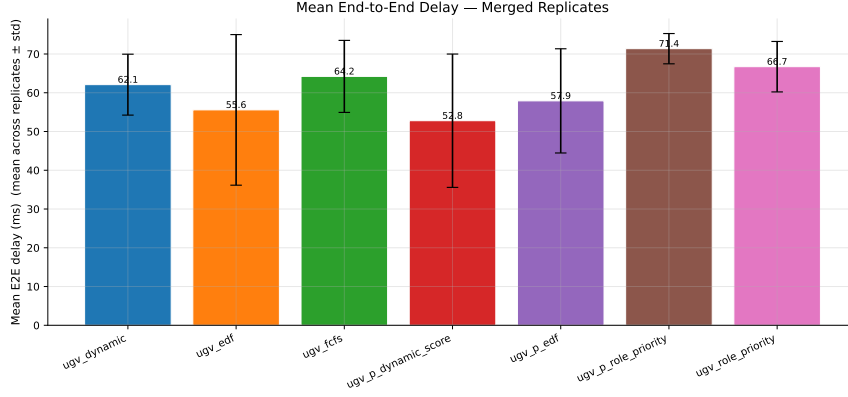


Figure 4.23. Mean end-to-end delay by protocol.

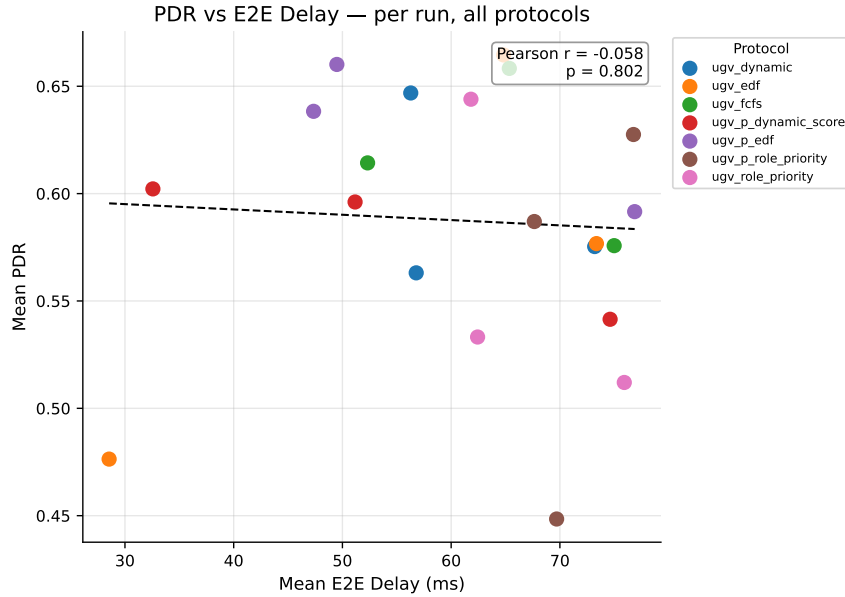


Figure 4.24. Joint view of PDR and mean end-to-end delay.

The exact definition of the bias-corrected delay metric used in the robustness check is provided in Appendix E.5; the base delay definition remains in Appendix E.1.

The conditioning effect is illustrated by Fig. 4.25, while Fig. 4.26 shows the corresponding robustness check.

4.6 Controlled Validation Under Fixed Weather

To validate the main RQ2 findings under a controlled environment, we ran Test Round 3 with fixed sunny weather. This removes weather-driven routing losses and isolates the effect of charging protocols on performance. We compare 7 protocols (3 replicates each, 21 runs total). The key question is whether the crosslayer

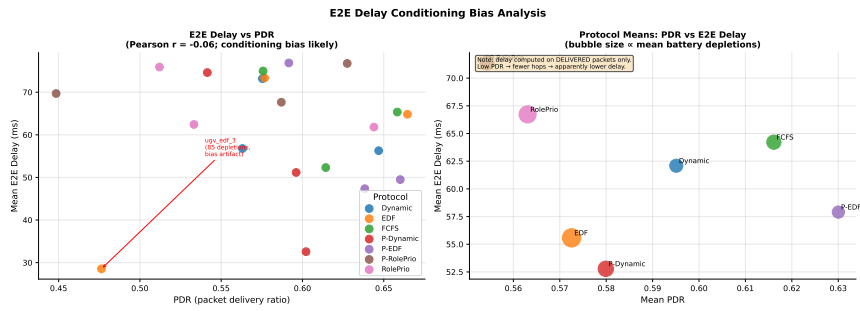


Figure 4.25. Conditioning-bias analysis for end-to-end delay.

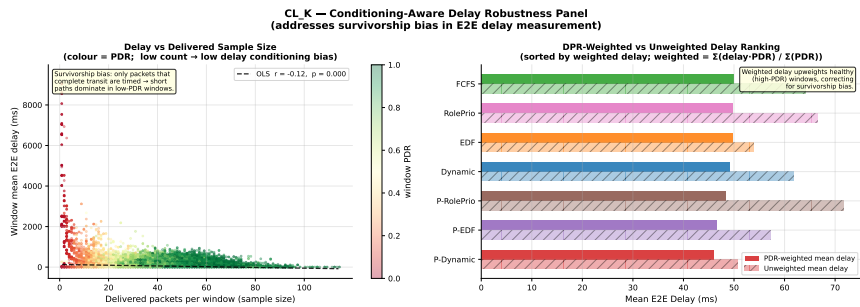


Figure 4.26. Robustness checks for delay interpretation.

relationships (charging success $\rightarrow$ survivability $\rightarrow$ network quality) still hold. In summary, Round 3 saw higher overall PDR and fewer route drops, but many relative trends persist: e.g. protocols with higher success rates still have higher PDR, while those with many depletions still have lower PDR, even though some rankings shifted due to the removal of weather effects

4.6.1 Rationale and Experimental Setup

4.6.2 Results and Comparison with Test Round 2

4.6.3 Interpretation

Chapter 5

Conclusions & Future Work

The grasping power of the mirror. .

Appendix Guide

Several parts of the simulator are specification-heavy (e.g., the **TrafficMessage** format, charging-decision payload fields, and the append-only logging schema). To preserve readability in the architecture chapter while ensuring full reproducibility, these technical details are moved to the appendices. The main text references the appendices whenever precise fields, equations, or configuration values are required.

Appendix A

Message and Protocol Specifications

A.1 TrafficMessage schema

All inter-agent communication in the simulator is represented as `uav_msgs/msg/TrafficMessage` instances published on the FANET bus. The message type encapsulates:

- **Addressing.** `src_id` and `dst_id` identify the logical end-points; `next_hop_id` identifies the immediate hop recipient.
- **Flow classification.** `flow_type` distinguishes DATA packets (0) from CONTROL packets (1). `control_type` is a string opcode (e.g., HEARTBEAT, ACK, DEPLOYMENT_CMD, MOTION_START, CHARGE_REQUEST, CHARGE_DECISION, RECOVERY_START, CLUSTER_REASSIGN, TASK_ASSIGN, NEW_DEPLOYMENT, MEMBER_FALLBACK, FAILURE_EVENT, DROP).
- **Hop accounting.** `hop_count` is incremented at each forwarding step; `ttl` acts as a hop limit (0 = unlimited). `recent_hops` is a bounded hop-history field used for loop detection.
- **Reliability.** `requires_ack` requests an application-layer ACK from the final destination. `ref_msg_id` links ACK and DROP messages back to the originating message.
- **Timestamps.** `creation_time` carries the producer's ROS clock at generation; `last_rx_time` is set by the last-hop receiver and used by the network monitor for end-to-end delay measurement.
- **Payload.** A free-form string field packs structured data; conventions used in the codebase include comma-separated tuples for deployment poses and semicolon-separated `key=value` pairs for charging decision rationales.

A.2 Charging decision payload specification

The UGV communicates scheduling outcomes using CHARGE_DECISION control messages encoded as `TrafficMessage(flow_type=1, control_type="CHARGE_DECISION")`. The payload is a semicolon-delimited set of `key=value` pairs:

- `accepted=1` / `accepted=0`: outcome of the decision.
- `policy`: the policy string that produced the decision.
- `priority`: the scheduling priority score assigned.
- `rank_index`: position in the sorted queue.
- `queue_size`: queue length at decision time.
- `tte_sec`: time-to-empty estimate used by the policy.
- `score`: composite score (dynamic policy only).
- `reason=PREEMPTED`: present when a preemption is issued.
- `target_action=STOP_CHARGING`: present in a preemption decision directed at the victim.

A.3 ROS 2 package and interface tables

Table A.1. ROS 2 package organisation.

Package	Responsibility
<code>uav_msgs</code>	Shared message, service, and action definitions.
<code>uav_fleet</code>	UAV behavioural model (CH and Member): mobility, energy, telemetry, FANET forwarding, charging FSM, ACK.
<code>ugv_charger</code>	UGV dock model, multi-policy scheduler, FANET decision routing.
<code>routing_manager</code>	Centralised routing table computation and reachability alerts.
<code>fault_injector</code>	Weather-driven packet drop between raw and processed FANET bus.
<code>weather_server</code>	Fixed or Markov weather regime generator.
<code>coverage_planner</code>	Deployment computation, task-point generation, motion-start barrier.
<code>sink_gateway</code>	Telemetry sink, delivery tap, ACK, CH status aggregation.
<code>ch_manager</code>	Cluster membership publication and recovery-driven update.
<code>recovery_manager</code>	Failure watchdog, CH re-election, FANET recovery control injection.
<code>network_monitor</code>	Omniscient experiment logger; CSV and JSON log output.
<code>system_bringup</code>	Launch file, run YAML configurations, task-point files.
<code>planner_viz</code>	Optional matplotlib visualiser (development/debug).

Table A.2. Principal node–interface mapping (P = publishes, S = subscribes).

Node	Publishes	Subscribes
uav_node	/fanet/status, /fanet/network_bus_raw, /fanet/delivered, /fanet/routing_event, /uav_fleet/charge_requests, /uav_fleet/failure_events	/fanet/network_bus, /fanet/status, /fanet/routing_table, /ch_manager/cluster_info, /environment/weather, /coverage_planner/task_points
ugv_charger_node	/fanet/network_bus_raw, /fanet/status, /fanet/delivered, /ugv/charge_decisions, /ugv/charging_snapshot, /ugv/queue_events	/fanet/network_bus, /fanet/status, /fanet/routing_table, /uav_fleet/failure_events
routing_manager_node	/fanet/routing_table, /routing_manager/alerts	/fanet/status, /fanet/routing_event
sink_gateway_node	/fanet/delivered, /fanet/network_bus_raw, /fanet/status, /fanet/routing_event	/fanet/network_bus, /fanet/routing_table, /coverage_planner/deployment
fault_injector_node	/fanet/network_bus, /fanet/delivered	/fanet/network_bus_raw, /environment/weather
recovery_manager_node	/fanet/network_bus_raw	/fanet/status, /fanet/network_bus, /routing_manager/alerts, /uav_fleet/failure_events, /coverage_planner/task_points, /ch_manager/cluster_info
network_monitor_node	/network_monitor/stats	/fanet/network_bus, /fanet/network_bus_raw, /fanet/delivered, /fanet/status, /fanet/routing_table, /uav_fleet/charge_requests, /ugv/charge_decisions, /ugv/queue_events, /ugv/charging_snapshot, /environment/weather

Appendix B

Algorithms and Mathematical Details

B.1 Deployment algorithms

B.1.1 Cluster-head placement from task points (k-means)

Once task points are available, the planner partitions them into NCH clusters using a k-means-style assignment/update loop run for 15 iterations. k-means alternates between two steps: an assignment step, in which each task point is assigned to the nearest cluster centroid (in Euclidean distance), and an update step, in which each centroid is recomputed as the arithmetic mean of all points currently assigned to it. The two steps are repeated until convergence or the iteration budget is exhausted. k-means is appropriate here because the objective - minimise the total squared distance from each task point to its serving CH - directly corresponds to minimising coverage path lengths and telemetry relay distances within each cluster. After individual CH poses are computed, a connectivity refinement pass (`tightenChConnectivity`) is applied to ensure sufficient CH-CH overlap and to guarantee that the sink node lies within at least one CH's communication range.

B.1.2 Connectivity refinement

`tightenChConnectivity` is a post-processing step that adjusts cluster-head positions so the CH network stays reliably connected while still respecting the task-driven layout. It iteratively pulls CHs toward nearby neighbors (to keep overlap/communication links valid), nudges them back toward anchor positions (to preserve mission intent), and ensures at least one CH remains within communication range of the sink. In short, its main purpose is to balance connectivity robustness with layout fidelity and sink reachability before deployment/routing continues.

B.1.3 UGV placement via geometric median (Weiszfeld algorithm)

The sink gateway is fixed at the origin $(0,0)$ of the planner coordinate frame and is assigned role 2. The UGV is placed at the geometric median of the computed CH positions, approximated via a Weiszfeld-style iterative procedure, and is then displaced toward the nearest CH if necessary to remain within communication range; it is assigned role 3. The geometric median minimises the sum of Euclidean distances

from a query point to a set of input points,

$$\mathbf{m}^* = \arg \min_{\mathbf{p}} \sum_{i=1}^{N_{\text{CH}}} \|\mathbf{p} - \mathbf{c}_i\|_2 \quad (\text{B.1})$$

where $\mathbf{c}_i$ are the CH positions. Unlike the arithmetic mean (which minimises the sum of *squared* distances and is therefore sensitive to outliers), the geometric median is robust to the case where one or two CHs are deployed far from the main cluster - a realistic scenario when CHs are spread to cover a wide disaster area. The Weiszfeld algorithm solves (B.1) iteratively via the fixed-point update

$$\mathbf{p}^{(t+1)} = \frac{\sum_{i=1}^{N_{\text{CH}}} \frac{\mathbf{c}_i}{\|\mathbf{p}^{(t)} - \mathbf{c}_i\|_2}}{\sum_{i=1}^{N_{\text{CH}}} \frac{1}{\|\mathbf{p}^{(t)} - \mathbf{c}_i\|_2}} \quad (\text{B.2})$$

which is a weighted centroid update in which closer CH positions receive higher weight. The update converges to the true geometric median from any initialisation that is not itself one of the input points, and in practice reaches a stable solution within tens of iterations. Placing the UGV at the geometric median minimises the worst-case travel distance for CHs returning to recharge.

B.2 Weather impairment and energy drain models

B.2.1 Packet-drop model in the fault injector

$$p_{\text{drop}} = \min(p_0 + a_w \cdot w + a_r \cdot r + a_t \cdot |T - T_{\text{ref}}|, p_{\text{max}}) \quad (\text{B.3})$$

where w is wind speed, r is rain intensity, T is temperature, and p_0 , a_w , a_r , a_t , p_{max} are configurable parameters.

Separate multipliers (`drop_control_multiplier`, `drop_data_multiplier`) allow control and data traffic to be impaired at different rates.

B.2.2 Weather-dependent energy drain model

Each UAV's energy consumption rate is computed at every status publication cycle as a product of four factors:

$$\text{drain_rate} = \text{base_drain} \times f_{\text{temp}} \times f_{\text{wind}} \times f_{\text{rain}} \quad (\text{B.4})$$

where `base_drain` is a role-dependent baseline (cluster-heads draw at rate r_{CH} , members at r_{m}); f_{temp} is a piecewise-linear temperature multiplier ranging from 1.7 at sub-zero conditions down to 1.0 in the 15–30 °C comfort band and up to 1.4 above 40 °C; f_{wind} is a directional wind penalty

$$f_{\text{wind}} = \max\left(0.6, 1 + k_{\text{wind}} \cdot \cos \theta \cdot \frac{v_{\text{wind}}}{v_{\text{ref}}}\right), \quad k_{\text{wind}} = 0.5, \quad v_{\text{ref}} = 10 \text{ m/s} \quad (\text{B.5})$$

with θ the angle between UAV heading and headwind direction; and f_{rain} is a rain-intensity penalty

$$f_{\text{rain}} = 1 + k_{\text{rain}} \cdot r_{\text{norm}}, \quad k_{\text{rain}} = 0.4 \quad (\text{B.6})$$

where $r_{\text{norm}} \in [0, 1]$ is rainfall normalised from 0–20 mm/h, capping the rain overhead at +40%.

Appendix C

Parameter Tables and Calibration Details

C.1 Time compression and parameter mapping

C.1.1 Time-compression factor

Experiments are executed for a wall-clock simulation horizon of three hours, yet are designed to represent the operational dynamics of a 24-hour disaster mission. To bridge these two timescales, a time-compression factor

$$s = \frac{24}{3} = 8 \tag{C.1}$$

is applied: one second of simulation time corresponds to eight seconds of effective mission time. The motivation is purely methodological - running a full 24-hour wall-clock experiment is impractical for a controlled, reproducible evaluation, whereas a three-hour run is long enough to observe the long-horizon phenomena of interest: queue buildup under different scheduling policies, repeated charging cycles, cluster-head failures and recoveries, and multiple weather-phase transitions driven by the Markov chain (Section 3.2.3). No physical claim about UAV aerodynamics or communication propagation is implied by this scaling.

C.1.2 Core simulation parameters

C.1.3 Implied effective endurance

The chosen capacity and drain-rate combination maps to the following simulated and effective endurance values, which are consistent with published manufacturer specifications [27]:

- **Member UAV.** $77.0 / 0.223192 \approx 345 \text{ s} \approx 5.75 \text{ min}$ simulation time; multiplied by $s = 8$ gives an effective mission endurance of approximately **46 min** - in line with the 40–46 min range of the Mavic 3 Classic under controlled conditions.
- **Cluster-head UAV.** $115.2 / 0.279272 \approx 412 \text{ s} \approx 6.9 \text{ min}$ simulation time; effective mission endurance of approximately **55 min**, consistent with enterprise-class platforms carrying larger battery packs.

Table C.1. Core simulation parameters used across all experiments. Battery and drain values are from `ugv_dynamic.yaml`; platform capacities are grounded against OEM specifications [27].

Parameter	Value	Description
<i>Battery and energy</i>		
Member battery capacity	77.0 Wh	Prosumer-class reference (Mavic 3 Classic)
CH battery capacity	115.2 Wh	Larger-frame platform
Member drain rate	0.223192 Wh/s	At $8\times$ time compression
CH drain rate	0.279272 Wh/s	Relay overhead included
textttbattery_threshold	30%	Fallback threshold (UGV range unknown)
textttugv_reserve_energy	10.0 Wh	Safety reserve margin in request threshold
ugv_buffer_energy	10.0 Wh	Extra buffer margin in request threshold
Time-compression factor	$8\times$	3 h sim $\equiv$ 24 h mission
<i>Network and routing</i>		
network.comm_radius_m	400 m	Effective UAV–UAV link radius
routing_manager.recompute_period_sec	configurable	Route recomputation period
routing_manager.status_timeout_sec	configurable	Status freshness threshold
<i>UGV charger</i>		
ugv.max_parallel_spots	3	Dock capacity (parallel slots)
ugv.charger_power_w	set by launch	Charging power (W)
<i>Experiment control</i>		
global.experiment_timeout_s	10800 s	3-hour run wall-clock duration
global.rng_seed	fixed per sweep	Master RNG seed

Appendix D

Logging Schema and Metric Extraction Details

D.1 Logging schema

The network monitor produces the following artefacts, all written under `<output_dir>/<run_id>/`:

Event logs (one row per discrete event, append-only):

- `packet_generated_events.csv`: one row per unique packet observed on `/fanet/network_bus_raw`, recording `msg_id`, flow type, control type, source, destination, creation timestamp, and payload size.
- `packet_delivered_events.csv`: one row per successfully delivered packet observed on `/fanet/delivered`, recording delivery timestamp, receiver, hop count, and TTL at delivery.
- `packet_drop_events.csv`: one row per DROP message, recording the original `ref_msg_id`, drop reason, and dropper node.
- `charge_request_events.csv`: one row per new `CHARGE_REQUEST`, recording UAV ID, role, battery at request, and request timestamp.
- `charge_decision_events.csv`: one row per delivered `CHARGE_DECISION`, recording the request back-reference, outcome, and decision latency.
- `charge_session_events.csv`: one row per charging lifecycle transition (`DOCK_START`, `DOCK_END`, `PREEMPTED`, `TIMEOUT`, `ENERGY_DEPLETED`), recording waiting time, charge duration, energy transferred, and battery levels.
- `recovery_events.csv`: one row per recovery control message observed on the FANET bus, recording the epoch, member/CH IDs, and deployment coordinates where applicable.
- `death_events.csv`: one row per battery-dead UAV, recording time, role, battery at death, and estimated surviving fleet size.
- `preemption_events.csv`: one row per preemption (populated from `/ugv/queue_events`), recording victim and winner IDs, roles, priorities, and policy.

Time-series logs (one row per node per sampling period):

- **status_timeseries.csv**: per-UAV battery level, charging state, role, position, and energy consumption rate at 1 Hz.
- **weather_timeseries.csv**: global weather regime and numeric fields at 1 Hz.
- **charge_queue_timeseries.csv**: queue length (total and by role), active sessions, dock utilisation, mean waiting times, and fleet activity counters at 1 Hz.
- **network_timeseries.csv**: sliding-window PDR, delay, jitter, and per-category charging-control PDR at 1 Hz.

Aggregated snapshots:

- **qos_metrics.csv**: cumulative QoS statistics per (flow_type, control_type) category, flushed every csv_write_period_sec.
- **messages.csv**: per-message lifecycle summary (generated, delivered, dropped, delay, hop count), flushed incrementally.
- **charge_events.csv**: per-charging-request end-to-end record (outcome, all timing fields, energy recovered, decision rationale parsed from payload), exported once the outcome reaches a terminal state.
- **summary_snapshots.jsonl**: one JSON object per flush capturing run-level KPIs (charging outcomes, network QoS, recovery event counts).

D.2 Metric extraction details

Chapter 4 derives all experimental metrics from the above artefacts through a post-processing pipeline. In brief:

- **Packet delivery ratio** is computed from `packet_generated_events.csv` and `packet_delivered_events.csv` joined on (run_id, run_instance_id, msg_id), using the last row of `qos_metrics.csv` per run as a cross-check.
- **End-to-end delay** is the difference between `delivered_time_s` (from the delivered events file) and `creation_time_s` (from the generated events file).
- **Charging waiting time** are extracted from `charge_session_events.csv` rows with `event_type=DOCK_START`, grouped by role and policy.
- **Network vs. charging interaction** is studied by joining `charge_events.csv` (which embeds decision-time control PDR in `decision_ctrl_pdr`) with `network_timeseries.csv`.
- **Preemption rates** are normalised by run duration using `preemption_events.csv` and the final `t_rel_s` from `summary_snapshots.jsonl`.

Appendix E

Data analysis details

E.1 Metric Definitions

This appendix collects the metric definitions used in Chapter 4. Unless otherwise stated, all protocol-level comparisons are computed per run first and then aggregated across the three replicates of the same protocol.

E.1.1 Network metrics

- **Packet delivery ratio (PDR).** For each run,

$$\text{PDR} = \frac{\sum \text{delivered}}{\sum \text{generated}}.$$

The numerator and denominator are taken from `qos_metrics.csv`.

- **Window PDR.** The time-varying reliability measure `window_pdr` is read from `network_timeseries.csv` after removing invalid negative sentinel values.
- **Mean end-to-end delay.** For each run, mean delay is computed from valid values of `window_delay_mean_ms` in `network_timeseries.csv`. Because this quantity is defined only over delivered packets, it is interpreted cautiously in the main text.

E.1.2 Charging metrics

- **Charge success rate.**

$$\text{Success Rate} = \frac{N_{\text{STARTED}}}{N_{\text{requests}}}.$$

- **Timeout rate.**

$$\text{Timeout Rate} = \frac{N_{\text{TIMEOUT}}}{N_{\text{requests}}}.$$

- **ROUTING_DROP rate.**

$$\text{RoutingDrop Rate} = \frac{N_{\text{ROUTING_DROP}}}{N_{\text{requests}}}.$$

- **Decision latency.** Mean or distribution of valid `decision_latency_ms` values in `charge_events.csv`. Rows with sentinel value -1 are excluded.
- **Effective wait.** Mean or distribution of valid `effective_wait_ms` values in `charge_events.csv`. Rows with sentinel value -1 are excluded.
- **Energy recovered per session.** Distribution of valid `energy_charged_wh` values in `charge_session_events.csv`; incomplete sessions are excluded.
- **Dock utilisation.** Mean of valid `ugv_dock_utilization` samples in `charge_queue_timeseries.csv`.

E.1.3 Survivability metrics

- **Total depletion events.** Number of rows in `death_events.csv` for a run.
- **Cluster-head and member depletions.** The same count split by the `role` field, where 1 denotes a cluster head and 0 a member UAV.

E.2 Aggregation and Statistical Summaries

All aggregate protocol comparisons in Chapter 4 follow the same pattern: (1) compute one scalar summary per run; (2) group the three replicate runs of the same protocol; (3) report the protocol mean and standard deviation.

E.2.1 Mean and standard deviation

For a protocol with replicate-level values x_1, x_2, x_3 , the reported mean is

$$\bar{x} = \frac{1}{3} \sum_{i=1}^3 x_i,$$

and the reported standard deviation is

$$s = \sqrt{\frac{1}{n-1} \sum_{i=1}^3 (x_i - \bar{x})^2}, \quad n = 3.$$

E.2.2 Coefficient of variation

When workload consistency is discussed, the coefficient of variation is used:

$$\text{CV} = \frac{s}{\bar{x}} \times 100\%.$$

This is applied to per-run generated-packet counts.

E.2.3 Correlation summaries

Where Chapter 4 reports Pearson or Spearman coefficients, each run contributes one scalar value per metric. Pearson correlation is used for linear association and Spearman correlation for rank-based monotonic association.

E.3 Windowing and Time-Series Processing

The time-series figures in Chapter 4 are derived from `network_timeseries.csv`, `charge_queue_timeseries.csv`, `charge_events.csv`, `death_events.csv`, and `weather_timeseries.csv`.

E.3.1 Valid-sample filtering

Negative values in `window_pdr`, `window_delay_mean_ms`, and other logged fields are interpreted as sentinel placeholders and excluded before plotting or aggregation. Similarly, -1 timing values in charging logs are treated as missing.

E.3.2 Merged time-series plots

For merged protocol plots, replicate time series are interpolated onto a shared time grid before computing the mean trajectory. Missing values are ignored in the pointwise average.

E.3.3 Weather joins

Weather-conditioned plots are obtained by assigning each network sample the nearest weather regime in time, using the shared `t_rel_s` axis.

E.3.4 Event binning

For event-aligned plots, event timestamps are cleaned by removing missing, negative, and out-of-range values. Event counts are then aggregated into fixed-width time bins for visual comparison with PDR and dock-utilisation trajectories.

E.4 PDR Dip Threshold for Event-Aligned Plots

The causal-analysis figures use a simple rule to identify PDR dip intervals. For a given curve, the threshold is defined as the 10th percentile of all valid (non-negative) `window_pdr` values. Any contiguous interval below this threshold is marked as a dip interval. On merged plots, the percentile is computed on the merged curve rather than separately for each replicate.

E.5 Delay Robustness and Bias-Corrected Delay

The mean end-to-end delay reported by the network monitor is conditioned on the set of packets that were successfully delivered. Under severe attrition, the delivered set shrinks and tends to contain shorter or simpler paths, which can make a degraded network appear artificially fast.

To reduce this effect, Chapter 4 uses a PDR-weighted delay only as a robustness check. For a protocol with valid time windows indexed by k ,

$$\text{Weighted Delay} = \frac{\sum_k d_k p_k}{\sum_k p_k},$$

where d_k is `window_delay_mean_ms` and p_k is the corresponding `window_pdr`. This weighting downweights low-PDR windows, where raw delay is most likely to be distorted by delivery conditioning.

E.6 Dataset Validation and Preprocessing Notes

This appendix summarises the validation checks and preprocessing rules applied to the Test Round 2 dataset before the comparative analysis reported in Chapter 4.

E.6.1 Dataset Completeness and Structural Consistency

The Test Round 2 dataset contains all seven target protocols, with three replicate runs per protocol, for a total of 21 run folders. For each run, the key analysis files required in this chapter were present, namely `qos_metrics.csv`, `network_timeseries.csv`, `charge_events.csv`, `death_events.csv`,

`charge_session_events.csv`, and `charge_queue_timeseries.csv`. All runs also reached the expected mission duration of 180 minutes.

These checks indicate that the comparative dataset is structurally complete and that no protocol is disadvantaged by missing logs or shortened execution time.

E.6.2 Column-Level Caveats and Sentinel Handling

Some files require light interpretation before aggregation. In `death_events.csv`, the `role` field is encoded numerically rather than as explicit labels, with 1 denoting cluster heads and 0 denoting member UAVs. In `charge_events.csv`, `decision_latency_ms` may take the sentinel value -1 when no valid decision timing exists, for example when a request does not successfully reach the scheduler. Similarly, in `charge_session_events.csv`, fields such as charge duration or recovered energy may remain at -1 for sessions that have not yet completed.

In addition, early windows in `network_timeseries.csv` may contain negative sentinel values before sufficient packets have accumulated for a stable estimate. These entries are treated as invalid placeholders and excluded from aggregate statistics and time-series plots.

E.6.3 Workload Comparability Across Protocols

A fair protocol comparison requires comparable offered workload across runs. Overall, packet generation in Test Round 2 is highly consistent for most protocols, with only limited variation between replicates. The few visibly lower-generation runs are associated with runs that also exhibit heavier UAV depletion, suggesting that the reduced traffic is a consequence of fleet attrition rather than inconsistent scenario configuration.

This distinction is important for interpretation. It means that differences in network outcomes should not be read as the product of substantially different input load, but rather as a consequence of how well each policy preserves an operational fleet under the same nominal scenario.

E.6.4 Duplicated Windows and Non-Corrupt Anomalies

Some runs contain duplicated timestamps in `network_timeseries.csv`, with two rows recorded for the same window. Inspection shows that these entries carry the same metric values and therefore do not indicate conflicting measurements. They are interpreted as repeated callback emissions from the monitoring pipeline rather than corruption of the underlying data.

Likewise, prolonged low `alive_count` values or repeated depletion events in specific runs are treated as behavioural outcomes of the simulator, not as logging

errors. These cases are analysed later as protocol-specific instability or failure cascades rather than removed during preprocessing.

E.6.5 Role-Stratified Charging KPI Computation**E.6.6 Cross-CH Request Synchronization Analysis**

Bibliography

- [1] Regione Emilia–Romagna. Alluvione in emilia-romagna di maggio 2023. Geoportale Regione Emilia-Romagna, 2023.
- [2] Indu Chandran and Kizheppatt Vipin. Multi-uav networks for disaster monitoring: Challenges and opportunities from a network perspective. *Drone Systems and Applications*, 2024.
- [3] Matteo Prata Novella Bartolini, Andrea Coletta and Camilla Serino. On connected deployment of delay-critical fanets. *IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, pages 9720–9727, 2021.
- [4] Prakash Ranganathan Mitch Campion and Saleh Faruque. Uav swarm communication and control architectures: A review. *Journal of Unmanned Vehicle Systems*, pages 93–106, 2018.
- [5] Nikhil Paliwal Chathuranga M. Wijerathna Basnayaka Dushantha Nalin K. Jayakody Hwang-Cheng Wang P. Muthuchidambaranathan Abhishek Sharma¹, Pankhuri Vanjani. Communication and networking technologies for uavs: A survey. *Journal of Network and Computer Applications*, 2020.
- [6] Anand Srivastava Mansi Peer, Vivek Ashok Bohara. Multi-uav placement strategy for disaster-resilient communication network. *IEEE 92nd Vehicular Technology Conference (VTC2020-Fall)*, 2020.
- [7] Yifeng Zhou Jun Li and Louise Lamont. Communication architectures and protocols for networking unmanned aerial vehicles). *IEEE Globecom Workshops (GC Wkshps)*, 2013.
- [8] JianKang Zhang Jingjing Wang Chaoxing Yan, Lingang Fu. A comprehensive survey on uav communication channel modeling. *IEEE Access*, 7, 2019.
- [9] Omar Sami Oubbati, Abderrahmane Lakas, Fen Zhou, Mesut Günes, and Mohamed Bachir Yagoubi. A survey on position-based routing protocols for Flying Ad hoc Networks (FANETs). *Vehicular Communications*, 10:29–56, 2017.
- [10] Asanka Perera Isuru Munasinghe and Ravinesh C. Deo. A comprehensive review of uav-ugv collaboration: Advancements and challenges. *JSAN*, 2024.
- [11] Ahmet Harun Eker, Ahmet Öncü, and H. Isil Bozma. Rendezvous scheduling for charging coordination between aerial robot - mobile ground robot. In *2022 IEEE 18th International Conference on Automation Science and Engineering (CASE)*, 2022.

- [12] Nare Karapetyan, Ahmad Bilal Asghar, Amisha Bhaskar, Guangyao Shi, Dinesh Manocha, and Pratap Tokekar. Ag-cvg: Coverage planning with a mobile recharging ugv and an energy-constrained uav. arXiv:2310.07621, 2023.
- [13] Tully Foote Steven Macenski and William Woodall Brian Gerkey, Chris Lalancette. Robot operating system 2: Design, architecture, and uses in the wild. *Science Robotics*, 7(66):eabm6074, 2022.
- [14] Open Robotics. Concepts - ROS 2 Documentation (Kilted). <https://docs.ros.org/en/kilted/Concepts.html>. Accessed: 2026-03-01.
- [15] Open Robotics. Basic Concepts - ROS 2 Documentation (Kilted). <https://docs.ros.org/en/kilted/Concepts/Basic.html>. Accessed: 2026-03-01.
- [16] Open Robotics. Nodes - ROS 2 Documentation (Kilted). <https://docs.ros.org/en/kilted/Concepts/Basic/About-Nodes.html>. Accessed: 2026-03-01.
- [17] Open Robotics. Topics - ROS 2 Documentation (Kilted). <https://docs.ros.org/en/kilted/Concepts/Basic/About-Topics.html>. Accessed: 2026-03-01.
- [18] Open Robotics. Services - ROS 2 Documentation (Kilted). <https://docs.ros.org/en/kilted/Concepts/Basic/About-Services.html>. Accessed: 2026-03-01.
- [19] Open Robotics. Actions - ROS 2 Documentation (Kilted). <https://docs.ros.org/en/kilted/Concepts/Basic/About-Actions.html>. Accessed: 2026-03-01.
- [20] Open Robotics. Parameters - ROS 2 Documentation (Kilted). <https://docs.ros.org/en/kilted/Concepts/Basic/About-Parameters.html>. Accessed: 2026-03-01.
- [21] Open Robotics. Understanding nodes - ROS 2 Documentation (Kilted). <https://docs.ros.org/en/kilted/Tutorials/Beginner-CLI-Tools/Understanding-ROS2-Nodes/Understanding-ROS2-Nodes.html>. Accessed: 2026-03-01.
- [22] Open Robotics. Executors - ROS 2 Documentation (Kilted). <https://docs.ros.org/en/kilted/Concepts/Intermediate/About-Executors.html>. Accessed: 2026-03-01.
- [23] Open Robotics. Different ROS 2 middleware vendors - ROS 2 Documentation (Kilted). <https://docs.ros.org/en/kilted/Concepts/Intermediate/About-Different-Middleware-Vendors.html>. Accessed: 2026-03-01.
- [24] Data Distribution Service (DDS), Version 1.4. <https://www.omg.org/spec/DDS/1.4/PDF>, 2015. Accessed: 2026-03-01.
- [25] Open Robotics. Internal ROS 2 interfaces - ROS 2 Documentation (Kilted). <https://docs.ros.org/en/kilted/Concepts/Advanced/About-Internal-Interfaces.html>. Accessed: 2026-03-01.
- [26] Open Robotics. ROS 2 middleware implementations - ROS 2 Documentation (Kilted). <https://docs.ros.org/en/kilted/Concepts/Advanced/About-Middleware-Implementations.html>. Accessed: 2026-03-01.

- [27] DJI. DJI Mavic 3 Classic — specifications. <https://www.dji.com/mavic-3-classic/specs>, 2024. Accessed: 2025.
- [28] Utkarsh Ahuja et al. High-speed Wi-Fi systems for long range FANETS: Real problems, experiments, and lessons learnt. In *Proceedings of UASG 2021: Wings 4 Sustainability*, Lecture Notes in Civil Engineering. Springer, 2023. Discusses experimental Wi-Fi FANET link performance for medium to small UAVs in geospatial applications.
- [29] David Clerigues, Jamie Wubben, Carlos T. Calafate, Juan-Carlos Cano, and Pietro Manzoni. Enabling resilient UAV swarms through multi-hop wireless communications. *EURASIP Journal on Wireless Communications and Networking*, 2024, 2024. States that UAVs can communicate up to several hundred meters using Wi-Fi in open areas, necessitating relay strategies for larger coverage.
- [30] DJI. DJI Dock 2 — specifications. <https://enterprise.dji.com/dock-2/specs>, 2024. Charging time 20%–90% measured at 25 °C: 32 minutes. Accessed: 2025.
- [31] DJI. DJI FlyCart 30 — specifications. <https://www.dji.com/flycart-30/specs>, 2024. Max payload: 30 kg (dual battery), 40 kg (single battery). Accessed: 2025.